\

SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Accredited by NAAC A Grade

Assignment III

CO3: Apply scanning and enumeration techniques and engineer a reliable scan-data pipeline — architecture, validation, cleaning, reshaping, merging, automated file I/O and a reusable function library — on authorized, in-scope lab targets.
Evaluation note: This Pair-work assignment: each question has shared work plus identified Student A and Student B responsibilities. Award marks for a working, keyed data model, correct R operations with before/after evidence, reconciliation/validation checks, and defended joint answers — and verify that BOTH students' individual components are present. All scanning assumes an isolated lab (e.g. 192.168.56.0/24). Accept equivalent correct R.

Name of the Student : M SANA FATHIMA

Register No : 192371082

Department : CSE

Sub Code / Name : ITA1405 ETHICAL HACKING

Date of Submission : 08/11/2026

M SANA FATHIMA

Student Signature **Faculty Signature**

ITA14 ETHICAL HACKING

ASSIGNMENT 3 – CO3

Mode: Pair Work – **2 Students per Group**

Duration: 1 Week

Total Marks: 50

Pair-Work Principle

Both students are responsible for understanding the **complete assignment**. However, each question contains clearly identified individual responsibilities so that the contribution and understanding of each student can be independently verified.

Student A and Student B must not divide the assignment question-wise. Both students must participate in every question.

The individual responsibilities intentionally alternate across questions so that both students demonstrate evidence of:

Q1 – Investigate Requirements and Create the Scan-Data Architecture — 6 Marks

Shared Group Work

Together, the pair must:

- Together, translate the authorized-assessment scenario into service-scanning and enumeration requirements.
- Identify at least four scan datasets, such as Hosts, Ports, Services, OS/Versions and Enumeration findings.
- Create a schema and relationship plan showing the columns, data types and key fields (such as IP address and port) for each dataset.

- Formulate enquiry questions and define acceptance criteria for correctness, consistency, traceability and reproducibility of the scan data.

Student A – Host and Port Data Design Student
A must individually:

- Lead the requirements analysis and design the Hosts and Ports schemas, including fields, data types and key fields.
- Prepare the initial requirements-to-column mapping and justify the choice of scandata structure.

Student B – Service Relationship and Enquiry Design

Student B must individually:

- Lead the relationship map for Services, OS/Versions and Enumeration findings; identify the key fields that link them to hosts and ports.
- Formulate at least four testable enquiry questions and define acceptance criteria for the scan data.

Individual Verification

Faculty may ask Student A to explain the host and port data design, and Student B to explain the service relationships and enquiry questions.

Q2 – Create and Validate the Source Scan Datasets — 7 Marks

Shared Group Work
Together, the pair must:

- Together, construct realistic source scan datasets in R (hosts, ports, services and versions) and create appropriate data frames for each.
- Introduce controlled issues such as missing values, duplicate host-port records, inconsistent service names and out-of-scope addresses.
- Develop validation and standardization checks before the datasets are merged.

Student A – Scan-Data Construction and Defect Injection

Student A must individually:

- Construct the Hosts and Ports/Services data frames and add controlled test defects.
- Explain the intended schema and the defects introduced for testing.

Student B – Scan-Data Validation and Standardization Student B must individually:

- Construct the Services, OS/Versions and Enumeration data frames.
- Implement validation and standardization checks for required columns, valid IP and port ranges and consistent service names.

Joint Design Question

Together answer:

If you were designing a scan-data preparation and validation pipeline for a large authorized assessment, which validation checks would you apply first, and why?

Both students must be able to defend the answer.

Q3 – Build a Scan-Data Editing and Cleaning Module — 7 Marks

Shared Group Work

Together, the pair must:

- Together, develop reusable operations for missing values, inconsistent service names and duplicate host-port records.
- Show before-and-after data and test valid, invalid and boundary cases.
- Distinguish records that can be corrected automatically from records that require manual review.

Student A – Scan-Data Cleaning and Correction Rules Student A must individually:

- Lead cleaning operations for missing values, duplicate host-port records and malformed addresses.
- Explain the rules used for each automatic correction.

Student B – Service-Name Standardization and Test Validation
Student B must individually:

- Lead the rules for inconsistent service names and manual-review flags; design verification tests.
- Verify that cleaning does not silently remove meaningful scan findings.

Joint Conclusion

Together answer:

Why is automated scan-data cleaning alone insufficient to guarantee the accuracy of an assessment's findings?

Both students must be able to justify their answers with suitable examples from their datacleaning and validation tasks.

Q4 – Create a Scan-Result Reshaping Pipeline Using Melting and Casting — 7 Marks

Shared Group Work
Together, the pair must:

- Together, create a wide-format scan dataset (for example hosts as rows and ports as columns) and reshape it into long format (host, port, state).
- Create a cast operation for a host-by-service or subnet-by-service summary.
- Identify the identifiers, dimensions and measured variables and validate that the reshape preserves the information.

Student A – Scan-Result Reshaping and Transformation Student
A must individually:

- Implement the wide-to-long transformation and identify the identifier and measured columns.
- Explain why the selected long format suits the required analysis.

Both students must be able to justify their decision using theoretical complexity and experimental results.

Q5 – Design a Multi-Stage Scan-Data Merging Strategy — 6 Marks

Shared Group Work

Together, the pair must:

- Together, combine the Hosts, Ports, Services, OS/Versions and Enumeration datasets into a single master scan table.
- Identify the keys (such as IP address and port), select suitable merge types and test at least two alternatives.
- Detect unmatched records, duplicated matches and unexpected row multiplication after merging.

Student A – Host–Service Integration and Join Validation

Student A must individually:

- Lead the Services-to-Hosts and Ports-to-Hosts joins; document the key choices.
- Check the record counts and ensure the joins do not create unintended row multiplication.

Student B – Version–Enumeration Integration and Exception Analysis Student B must individually:

- Lead the OS/Versions and Enumeration integration; compare alternative join types.
- Help verify the final master scan table and document exceptions.

Cross-Understanding Requirement Student
B must be able to explain:
Why can different join types produce different record counts, and how can unmatched hosts or services be detected?
Student A must be able to explain:
What experimental evidence demonstrates that the Services-to-Hosts and Ports-to-Hosts joins are correct?
This deliberately tests understanding across the assigned responsibilities.

Q6 – Create an Automated Scan-File Reading and Writing Pipeline — 7 Marks

Shared Group Work
Together, the pair must:

Together, read source scan-output files (CSV or Nmap results exported to CSV), check the required columns and import them into data frames.

- Create a naming and versioning strategy that avoids accidentally overwriting earlier scan results.

Student A – Scan-File Import and Schema Validation Student
A must individually:

- Implement scan-file import and schema/type checks before processing.
- Demonstrate how the pipeline identifies missing required columns or unexpected formats.

Student B – Result Export, Versioning and Read-Back Validation Student
B must individually:

- Implement export of the cleaned, reshaped and master scan datasets; define the output naming and versioning.
- Compare the re-imported data with the written outputs.

DAY-4 DESIGN CHALLENGE

Q7 – Develop a Reusable Scanning and Enumeration Function Library — 10 Marks The Day-4 problem must be solved collaboratively.

Neither student may work on this question independently and simply combine the answers later.

Stage 1 – Independent Function Design Proposal

Student A – Validation and Cleaning Function Design Student A must individually:

- Lead the validation and cleaning functions for scan data; specify inputs, default arguments and return values.
- Test the functions independently and document their use.

Student B – Scan Transformation and File-Handling Function Design Student B must individually:

Lead the reshaping, merging and file-handling functions; include error handling.

- Demonstrate reuse with a different subnet or scan session.

Each student's initial proposal must be retained in the submission.

No marks are deducted merely because an initial design is incorrect.

The purpose is to provide evidence of independent algorithmic thinking.

Stage 2 – Integrated Pipeline Implementation and Validation

Student A – Import, Validation and Cleaning Integration Student A must individually:

- Integrate the import, validation and cleaning stages; record the input, rejected and cleaned record counts.
- Check that invalid inputs are handled consistently.

Student B – Transformation, Merging and Export Integration
Student B must individually:

- Integrate the reshaping, merging and export stages; record the merged and final record counts.
- Run the workflow again with the same inputs and check reproducibility.

Each student must document their implementation, test cases, record counts, and validation results.

Both students must demonstrate that the integrated pipeline operates correctly and produces consistent results.

The purpose is to assess independent implementation, integration accuracy, and reproducibility.

Stage 3 – Create and Validate a Reporting Scan Summary

Student B – Vulnerability-Exposure Reporting and Management Report Student B must individually:

- Create an exposure or service-version risk report and implement the summary.
- Explain how the report addresses a management requirement.

Stage 4 – Stress-Test, Refine and Optimize the Scan Pipeline

Student A – Dataset Scaling and Baseline Benchmarking Student A must individually:

- Generate or prepare scan datasets at the required scales (for example 50 and 5,000 host-port records) and benchmark the baseline.
- Record input size, elapsed time and correctness checks.

Student B – Bottleneck Analysis and Performance Optimization
Student B must individually:

- Analyse the measurements, identify a bottleneck and implement one refinement.

- Compare before-and-after performance and explain any limitations.

Stage 5 – Redesign the Pipeline Under a Changed Assessment Requirement

Student A – Schema Evolution and Architecture Redesign
Student A must individually:

- Analyse the impact on the schema, keys, validation and data retention. Propose revised data structures and rules for new scan sources such as UDP services or newly discovered hosts.
- Retain the impact analysis and explain the revised architecture.

Student B – Multi-Session Workflow Implementation and Change Validation Student B must individually:

- Implement or modify the multi-session scan/merge workflow and test retained, updated, unmatched and newly discovered hosts.
- Present test evidence, compare with the original pipeline and identify trade-offs.

Stage 6 – Joint Evaluation

Together determine:

- Why the scan-data pipeline requires a structured sequence of import, validation, cleaning, reshaping, merging and export operations.
- How modular functions and systematic transformations improve the correctness, maintainability and efficiency of the scanning workflow.
- What additional costs or challenges arise from validation, error handling, repeated scanning and data integration.
- Under what conditions the proposed pipeline might become inefficient, unreliable or unsuitable for changing assessment requirements.

Conclude:

- What characteristics should a security analyst consider when designing a reliable, efficient and maintainable scan-data pipeline for evolving networks?

CONTRIBUTION RECORD

Each pair must complete the following table at the end of the submission. The record must reflect actual work completed. Primary contribution does not mean exclusive responsibility.

Task	**Student A – Primary Contribution**	**Student B – Primary Contribution**	**Joint Work**
Q1			
Q2			
Q3			
Q4			
Q5			
Q6			
Q7			

INDIVIDUAL VERIFICATION

Faculty may select any task, including a component for which the student was not the primary contributor. Each student must therefore be able to:

- Explain the scan-data schema and key choices.
- Explain the cleaning, reshaping and merging decisions.
- Interpret the validation and reconciliation results.
- Explain the integrated pipeline and the scan-file read/write process.
- Defend the performance observations and design refinements.
- Respond to a changed input or assessment requirement.

A student should not answer, “My partner did this section.” Understanding of the complete submitted work is required.

NOTE: RECOMMENDED FACULTY VERIFICATION MODEL (VIVA)

For each student, conduct a brief 4–5 minute verification.

Question 1 – Primary responsibility:

Ask one technical question from the component the student led.

Question 2 – Partner's responsibility:

Ask one question from a component primarily completed by the partner.

Question 3 – Transfer question:

Change a condition and ask the student to reason about the consequences. For example: What happens if a product ID appears in Sales but not in Products? How should the pipeline handle a newly introduced product? What happens to processing time when transaction volume increases

Contents

Section	Title	Marks
0	Objectives, lab set-up and expected deliverables	-
Q1	Investigate requirements and create the scan-data architecture	6
Q2	Create and validate the source scan datasets	7
Q3	Build a scan-data editing and cleaning module	7
Q4	Create a scan-result reshaping pipeline (melting and casting)	7
Q5	Design a multi-stage scan-data merging strategy	6
Q6	Create an automated scan-file reading and writing pipeline	7
Q7	Day-4 design challenge: reusable scanning and enumeration function library (Stages 1-6)	10
-	Contribution record, individual verification (viva) preparation, conclusion	-
A-C	Appendices: Nmap acquisition, run order, figure index	-

0. Objectives, Lab Set-up and Expected Deliverables

0.1 Objective

Course Outcome 3 (CO3) requires the application of scanning and enumeration techniques and, more importantly, the engineering of a trustworthy data pipeline around the scan output. A port scan produces raw observations; an assessment is only as reliable as the data handling between the scanner and the final report. This report therefore designs, builds, tests and defends a pipeline with six stages - import, validate, clean, reshape, merge and export - and packages it as a reusable R function library.

0.2 Laboratory assumptions

- **Scope:** isolated host-only network 192.168.56.0/24 (a second lab subnet, 192.168.57.0/24, is used only to demonstrate reuse of the library).
- **Targets:** intentionally vulnerable lab virtual machines owned by the students (for example Metasploitable-type guests). Hostnames used in the datasets (web01, db01, mail01 and so on) are lab labels.

- **Software:** R 4.x with the reshape2 package; Nmap run from the lab attacker VM; results exported to CSV (Appendix A).
- **Datasets:** the scan datasets are realistic but small, hand-built data frames with deliberately injected defects, so that every validation, cleaning and merge decision can be verified by inspection.

0.3 Expected deliverables and outcomes

Q	Expected deliverable	Expected outcome / evidence	Where
Q1 (6)	Five scan datasets with keyed schema; relationship map; 4 enquiry questions; acceptance criteria	Normalised design (one row per host, one per host-port); keys host_id and (host_id, port, protocol); testable criteria for correctness, consistency, traceability, reproducibility	Tables 1.1-1.5, Figs 1.1-1.2
Q2 (7)	R data frames with injected defects; validation and standardisation checks; joint design answer	Defects detected and flagged, not deleted; scope-first validation order defended	Figs 2.1-2.3, Tables 2.1-2.2
Q3 (7)	Reusable cleaning operations; before/after data; valid, invalid and boundary tests; auto vs manual rules	Row counts reconciled; audit table of rejected rows; no open port silently lost	Figs 3.1-3.4, Tables 3.1-3.3
Q4 (7)	Wide-to-long melt; cast summaries; identifier/dimension/measure analysis; round-trip validation	Cast totals equal source totals (5 open ports, 7 scanned pairs)	Figs 4.1-4.2, Table 4.1
Q5 (6)	Master table from five datasets; two or more join types tested; unmatched and multiplied rows detected	Master has 7 rows = host-port rows; row multiplication demonstrated and guarded	Figs 5.1-5.3, Table 5.1
Q6 (7)	Import with schema/type checks; versioned export; read-back validation	Malformed files rejected or warned; no overwriting; lossless round trip	Figs 6.1-6.2, Table 6.1
Q7 (10)	Function library; integrated pipeline; reports; benchmark; redesign; joint evaluation	Counts 10 / 2 / 1 / 7 / 7 / 7; reproducible; 4.8x refinement; multi-session classification	Figs 7.1-7.6, Tables 7.1-7.5

Q1. Investigate Requirements and Create the Scan-Data Architecture (6 Marks)

1.1 Shared group work - from assessment scenario to requirements

Scenario. An authorised assessment must (i) identify which hosts in the in-scope subnet are alive, (ii) discover open TCP/UDP ports, (iii) identify the service and version behind each open port, (iv) fingerprint the operating system, and (v) record enumeration findings with a severity. These five needs translate directly into five datasets.

Table 1.1 Requirements translated into scan datasets

Requirement	Scan dataset	Source (lab)	Granularity
Which hosts are up?	Hosts	Nmap host discovery (-sn)	One row per host
Which ports are open?	Ports	Nmap port scan (-sS)	One row per host-port-protocol
What runs on the port?	Services	Nmap -sV	One row per host-port
What OS is it?	OS/Versions	Nmap -O	One row per host
What did enumeration find?	Enumeration findings	NSE scripts / manual checks	One row per host-port finding

1.2 Schema and relationship plan

Table 1.2 Schema, data types and key fields

Dataset	Columns (type)	Primary key	Foreign key
Hosts	host_id (chr), ip (chr), hostname (chr), status (factor: up/down)	host_id	-
Ports	host_id (chr), port (int), protocol (factor: tcp/udp), state (factor: open/closed/filtered/open|filtered)	(host_id, port, protocol)	host_id -> Hosts
Services	host_id (chr), port (int), service_name (chr), version (chr)	(host_id, port)	(host_id, port) -> Ports
OS/Versions	host_id (chr), os_name (chr), os_accuracy (int)	host_id	host_id -> Hosts

Dataset	Columns (type)	Primary key	Foreign key
Enumeration	host_id (chr), port (int), finding (chr), severity (factor: info/low/medium/high)	(host_id, port) in this lab*	(host_id, port) -> Ports

**If several findings are recorded for one port, they are first collapsed to the highest-severity finding (or a count) before the merge, otherwise the master table would multiply rows (see Q5).*

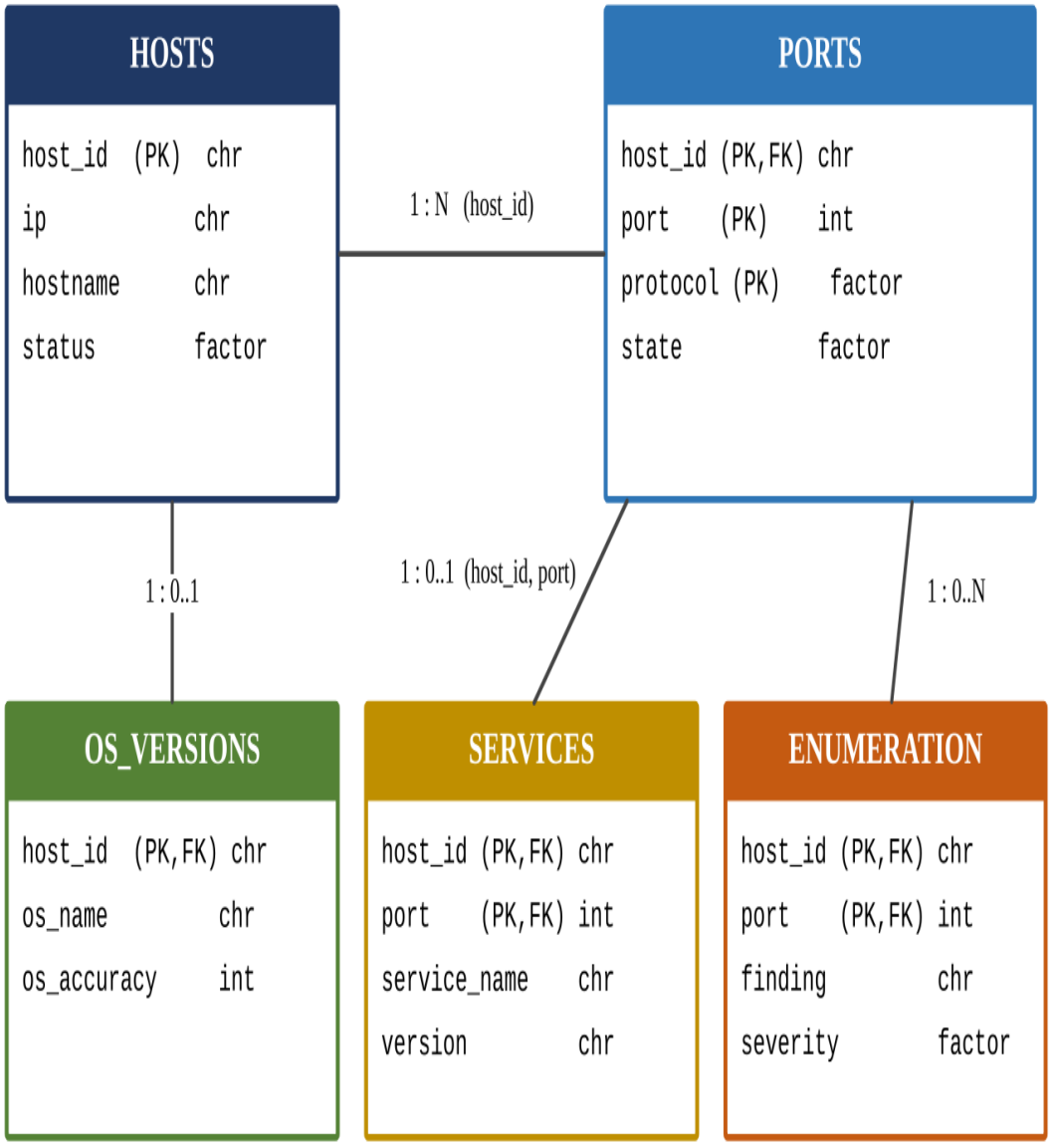


Figure 1.1 Entity-relationship plan (Hosts, Ports, Services, OS/Versions, Enumeration)

The schema is created in R so that column names and types are fixed before any data is loaded:

```
schemas <- list(
  Hosts       = data.frame(host_id=character(), ip=character(), hostname=character(),
                           status=factor(levels=c("up","down"))),
  Ports       = data.frame(host_id=character(), port=integer(),
```

```
                          protocol=factor(levels=c("tcp","udp")),
                          state=factor(levels=c("open","closed","filtered","open|filtered"))),
  Services    = data.frame(host_id=character(), port=integer(),
                          service_name=character(), version=character()),
  OS_Versions = data.frame(host_id=character(), os_name=character(), os_accuracy=integer()),
  Enumeration = data.frame(host_id=character(), port=integer(), finding=character(),
                          severity=factor(levels=c("info","low","medium","high")))
)
sapply(schemas, ncol)     # Hosts 4, Ports 4, Services 4, OS_Versions 3, Enumeration 4
```

1.3 Enquiry questions and acceptance criteria

Table 1.3 Testable enquiry questions

#	Enquiry question	Test (R expression)	Datasets
E1	Which open ports have no identified service?	master[master$state=="open" & is.na(master$service_name),]	Ports, Services
E2	Which hosts run an outdated service version?	master$version %in% eol_versions	Services, Hosts
E3	Are any scanned addresses out of scope?	!in_scope(hosts$ip)	Hosts
E4	Does every enumeration finding map to an open port?	merge(enum, ports, by=c("host_id","port"))$state == "open"	Enumeration, Ports

Table 1.4 Acceptance criteria

Criterion	Rule	How it is verified
Correctness	IPs are well formed (four octets, 0-255); ports are integers 1-65535; protocol and state belong to the allowed levels	valid_ip(), valid_port(), levels check; failures flagged
Consistency	Service names come from a controlled list (http, https, ssh, smb, smtp, mysql ...); one spelling per service	std_name() mapping; unmapped names flagged MANUAL
Traceability	Every finding, service and OS record resolves to a host_id (and port) in Hosts/Ports; every rejected row is kept in an audit table with a reason	anti-join tests; accounted rows = input rows
Reproducibility	Running the same inputs twice gives identical record counts and an identical master table; outputs are versioned, never overwritten	identical(a, b); timestamped file names

1.4 Student A - Host and port data design

Student A led the requirements analysis for Hosts and Ports. The design is **normalised**: a host appears once in Hosts and a host-port pair appears once in Ports, linked by host_id. A single wide table (one row per host with a column per port) was rejected because it repeats host attributes, cannot store protocol and state per port cleanly, and changes shape whenever a new port is scanned. The host_id (H1, H2 ...) is a surrogate key derived from the IP so that the address itself can be corrected without breaking links.

Table 1.5 Requirements-to-column mapping and justification

Requirement	Column	Justification
Which hosts are up?	Hosts.status	One status value per host; filter status == "up"
Which ports are open?	Ports.state == "open"	State is a property of the host-port pair, not of the host

Requirement	Column	Justification
Which service/version?	Services.service_name, version	Separate table because a port may be open with no identified service
Which hosts are in scope?	Hosts.ip + in_scope()	Scope is tested on the address, early in the pipeline
Which findings are serious?	Enumeration.severity	Ordered factor allows ranking in the exposure report

1.5 Student B - Service relationship map and enquiry design

Student B led the relationship map. Services, OS/Versions and Enumeration all link to Hosts through **host_id**. Services and Enumeration additionally link through **port**, so (host_id, port) is the join bridge from a raw open port to its service and its findings. OS/Versions is a host-level attribute and is therefore joined on host_id alone, which cannot multiply rows. Student B also formulated the four enquiry questions in Table 1.3 and the acceptance criteria in Table 1.4.

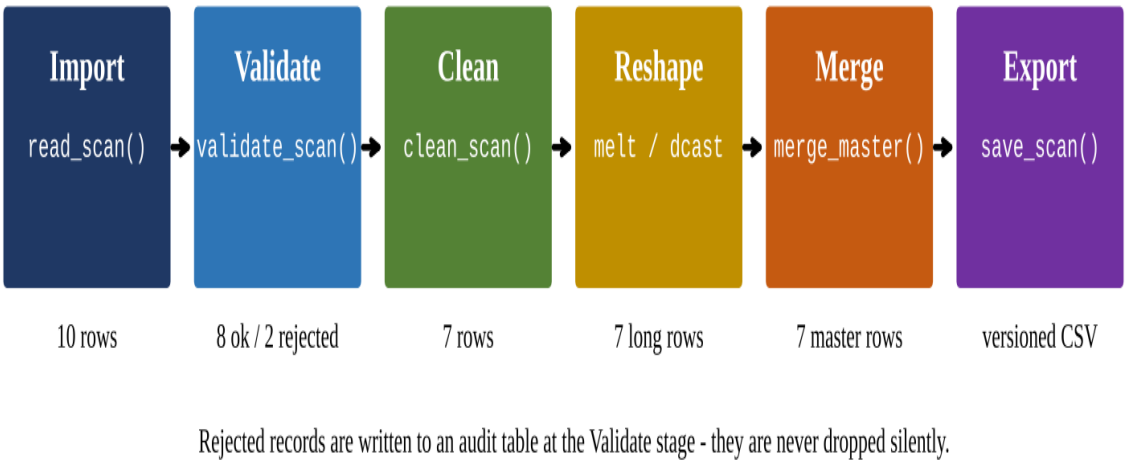


Figure 1.2 Scan-data pipeline architecture with record counts for scan session 1

1.6 Individual verification notes

- **Student A may be asked:** why is Ports keyed on three columns? (A host can expose the same port number on TCP and UDP; without protocol in the key the two would be treated as duplicates.)
- **Student B may be asked:** which key links a finding to its host? (host_id for the host, (host_id, port) for the open port.)

Q2. Create and Validate the Source Scan Datasets (7 Marks)

2.1 Shared group work - constructing the source datasets in R

Five data frames were constructed for the lab. Controlled defects were injected so that each validation rule has something to find. The hosts frame holds 7 rows, the ports frame 10 rows, services 6, OS/Versions 4 and enumeration 6.

```
hosts <- data.frame(
  host_id  = c("H1","H2","H3","H4","H1","H5","H6"),
  ip       = c("192.168.56.10","192.168.56.11","192.168.56.12","10.0.0.5",
               "192.168.56.10","192.168.56.13","192.168.56"),
  hostname = c("web01","db01",NA,"extsrv","web01","mail01","legacy"),
  status   = "up", stringsAsFactors = FALSE)


ports <- data.frame(
  host_id  = c("H1","H1","H1","H2","H2","H3","H3","H5","H5","H4"),
  port     = c(80, 80, 22, 3306, 70000, 445, 21, 25, 80, 8080),
  protocol = "tcp",
  state    = c("open","open","open","open","open","open","closed","open","filtered","open"),
  stringsAsFactors = FALSE)


services <- data.frame(
  host_id      = c("H1","H1","H2","H3","H5","H4"),
  port         = c(80, 22, 3306, 445, 25, 8080),
  service_name = c(" HTTP ","ssh","My-SQL","microsoft-ds","SMTP","www"),
  version      = c("Apache 2.4.18","OpenSSH 7.2","MySQL 5.5.62","Samba 4.3.11",NA,"Jetty 9.4"),
  stringsAsFactors = FALSE)


osv <- data.frame(host_id = c("H1","H2","H3","H5"),
  os_name = c("Ubuntu Linux 16.04","Ubuntu Linux 16.04","Ubuntu Linux 14.04","Debian Linux 9"),
  os_accuracy = c(96L,95L,91L,88L), stringsAsFactors = FALSE)


enum <- data.frame(host_id = c("H1","H1","H2","H3","H3","H5"),
  port = c(80, 22, 3306, 445, 21, 25),
  finding = c("Directory listing enabled","Weak SSH algorithms","Default account exposed",
              "SMBv1 enabled","Anonymous FTP login","SMTP VRFY enabled"),
  severity = c("medium","low","high","high","medium","low"), stringsAsFactors = FALSE)
```

```
Console ~/ITA14_Assignment3/
> print(hosts)
  host_id            ip hostname status
1      H1 192.168.56.10    web01     up
2      H2 192.168.56.11     db01     up
3      H3 192.168.56.12     <NA>     up
4      H4      10.0.0.5   extsrv     up
5      H1 192.168.56.10    web01     up
6      H5 192.168.56.13   mail01     up
7      H6    192.168.56   legacy     up
> print(ports)
   host_id  port protocol    state
1       H1    80      tcp     open
2       H1    80      tcp     open
3       H1    22      tcp     open
4       H2  3306      tcp     open
5       H2 70000      tcp     open
6       H3   445      tcp     open
7       H3    21      tcp   closed
8       H5    25      tcp     open
9       H5    80      tcp filtered
10      H4  8080      tcp     open
> print(services)
  host_id port service_name       version
1      H1   80         HTTP  Apache 2.4.18
2      H1   22           ssh   OpenSSH 7.2
3      H2 3306       My-SQL  MySQL 5.5.62
4      H3  445 microsoft-ds  Samba 4.3.11
5      H5   25          SMTP          <NA>
6      H4 8080           www     Jetty 9.4
```

Figure 2.1 R console: raw hosts, ports and services data frames with injected defects

Table 2.1 Defect register (controlled test defects)

ID	Dataset	Defect injected	Location	Expected detection
D1	Hosts	Missing hostname	H3 (row 3)	is.na(hostname)
D2	Hosts	Duplicate host record	H1 (rows 1 and 5)	duplicated(host_id, ip)
D3	Hosts	Out-of-scope address 10.0.0.5	H4 (row 4)	in_scope() = FALSE
D4	Hosts	Malformed address 192.168.56	H6 (row 7)	valid_ip() = FALSE
D5	Ports	Duplicate host-port record	H1:80 (rows 1 and 2)	duplicated(host_id, port, protocol)
D6	Ports	Invalid port number 70000	H2 (row 5)	valid_port() = FALSE

ID	Dataset	Defect injected	Location	Expected detection
D7	Services	Inconsistent names: " HTTP ", My-SQL, www, microsoft-ds, SMTP	rows 1, 3, 4, 5, 6	std_name() mapping
D8	Services	Missing version	H5:25	is.na(version) -> MANUAL
D9	Enumeration	Finding recorded for a closed port	H3:21	finding joined to state != open

2.2 Validation and standardisation checks

```
valid_ip <- function(x) {
  ok <- grepl("^(\\d{1,3}\\.){3}\\d{1,3}$", x)
  ok[ok] <- vapply(strsplit(x[ok], ".", fixed = TRUE),
                   function(o) all(as.integer(o) <= 255), logical(1))
  ok
}
in_scope    <- function(ip, scope = "^192\\.168\\.56(\\.|$)") grepl(scope, ip)
valid_port  <- function(p) !is.na(p) & p >= 1 & p <= 65535
std_service <- function(s) tolower(trimws(s))


check_cols <- function(df, req, label) {
  miss <- setdiff(req, names(df))
  if (length(miss)) stop(label, ": missing columns: ", paste(miss, collapse = ", "))
  invisible(TRUE)
}


check_cols(hosts, c("host_id","ip","hostname","status"), "hosts")
hosts_chk <- hosts
hosts_chk$flag_ip    <- ifelse(valid_ip(hosts_chk$ip), "ok", "MALFORMED")
hosts_chk$flag_scope <- ifelse(in_scope(hosts_chk$ip), "in", "OUT")
hosts_chk$flag_dup   <- ifelse(duplicated(hosts_chk[, c("host_id","ip")]), "DUP", "ok")
ports_chk <- ports
ports_chk$flag_port <- ifelse(valid_port(ports_chk$port), "ok", "BAD")
ports_chk$flag_dup  <- ifelse(duplicated(ports_chk[, c("host_id","port","protocol")]), "DUP",
"ok")
services_chk <- services; services_chk$service_std <- std_service(services$service_name)
```

```
Console ~/ITA14_Assignment3/
> check_cols(hosts, c("host_id","ip","hostname","status"), "hosts")    # passes
+   silently
> hosts_chk <- hosts
> hosts_chk$flag_ip     <- ifelse(valid_ip(hosts_chk$ip), "ok", "MALFORMED")
> hosts_chk$flag_scope <- ifelse(in_scope(hosts_chk$ip), "in", "OUT")
> hosts_chk$flag_dup    <- ifelse(duplicated(hosts_chk[, c("host_id","ip")]), "DUP",
+   "ok")
> print(hosts_chk)
  host_id            ip hostname status    flag_ip flag_scope flag_dup
1      H1 192.168.56.10     web01     up         ok         in       ok
2      H2 192.168.56.11      db01     up         ok         in       ok
3      H3 192.168.56.12      <NA>     up         ok         in       ok
4      H4       10.0.0.5   extsrv     up         ok        OUT       ok
5      H1 192.168.56.10     web01     up         ok         in      DUP
6      H5 192.168.56.13    mail01     up         ok         in       ok
7      H6     192.168.56    legacy     up MALFORMED         in       ok
> ports_chk <- ports
> ports_chk$flag_port <- ifelse(valid_port(ports_chk$port), "ok", "BAD")
> ports_chk$flag_dup  <- ifelse(duplicated(ports_chk[,
+   c("host_id","port","protocol")]), "DUP", "ok")
> print(ports_chk)
   host_id  port protocol     state flag_port flag_dup
1       H1    80      tcp      open        ok       ok
2       H1    80      tcp      open        ok      DUP
3       H1    22      tcp      open        ok       ok
4       H2  3306      tcp      open        ok       ok
5       H2 70000      tcp      open       BAD       ok
6       H3   445      tcp      open        ok       ok
7       H3    21      tcp    closed        ok       ok
8       H5    25      tcp      open        ok       ok
9       H5    80      tcp filtered        ok       ok
10      H4  8080      tcp      open        ok       ok
```

Figure 2.2 R console: validation flags on hosts and ports (rows are flagged, not deleted)

```
Console ~/ITA14_Assignment3/

> services_chk <- services; services_chk$service_std <-
+   std_service(services$service_name)
> print(services_chk[, c('service_name','service_std')])
  service_name  service_std
1         HTTP         http
2          ssh          ssh
3        My-SQL       my-sql
4 microsoft-ds microsoft-ds
5         SMTP         smtp
6          www          www
> check_cols(ports[, c("host_id","port")], c("host_id","port","protocol","state"),
+   "ports")
Error in check_cols(ports[, c("host_id", "port")], c("host_id", "port", "protocol",  :
  ports: missing columns: protocol, state
> sapply(list(hosts=hosts, ports=ports, services=services, osv=osv, enum=enum), nrow)
   hosts    ports services      osv     enum
       7       10        6        4        6
```

Figure 2.3 R console: service-name standardisation and required-column check

Reading the results. In Figure 2.2 the scope flag marks only H4 (10.0.0.5) as OUT, while the format flag marks only H6 (192.168.56) as MALFORMED. The scope pattern deliberately accepts 192.168.56 as being 'in the subnet' so that the malformed address is caught by the format check rather than disguised as an out-of-scope host. In the ports frame, row 2 is a duplicate and row 5 is an invalid port. Nothing has been removed yet: the checks only flag, which keeps the original evidence intact.

2.3 Student A - scan-data construction and defect injection

Student A constructed the Hosts and Ports/Services frames and injected defects D1-D8. The **intended schema** is Table 1.2: host_id and ip are character fields, port is numeric (integer after import), and state and protocol take a small set of levels. The defects were chosen so that every one maps to exactly one rule: D1 tests missing-value handling, D2 and D5 test key-based de-duplication, D3 tests the scope filter, D4 tests the address format check, D6 tests the port range check and D7 tests standardisation. Because each defect is isolated in a known row, a failed rule can be traced to one line of code.

2.4 Student B - scan-data validation and standardisation

Student B constructed the Services, OS/Versions and Enumeration frames and implemented the checks. **Required columns** are checked with stopifnot-style logic (check_cols), which stops before any processing if a column is missing - Figure 2.3 shows the error produced when protocol and state are absent. **IP and port ranges** are validated by valid_ip() (four octets, each 0-255) and valid_port() (1-65535, NA rejected). **Service names** are standardised by trimming whitespace and lower-casing; the controlled mapping (www to http, my-sql to mysql, microsoft-ds to smb) is applied in Q3.

2.5 Joint design question

If you were designing a scan-data preparation and validation pipeline for a large authorised assessment, which validation checks would you apply first, and why?

Table 2.2 Recommended validation order

Order	Check	Reason for the position
1	Required columns and schema	If a column is missing or mis-typed, every later check is meaningless; failing here costs almost nothing.
2	Scope filter (in-scope addresses only)	Scanning and storing data for out-of-scope targets is an authorisation problem, not just a quality problem. Removing those rows early also shrinks the data every later check must process.
3	Type and range checks (IP format, port 1-65535, valid state)	Cheap, row-by-row, decisive. Rows that fail cannot be repaired by rules and must be rejected or reviewed.
4	Standardisation (service names, case, whitespace)	Needed before duplicate detection and joins, otherwise 'HTTP' and 'http' are treated as different values.
5	Duplicate detection on the keys	Duplicates are only reliable after values are standardised and invalid rows removed; this is also the guard against row multiplication in joins.

The ordering is scope-first because schema and scope failures invalidate everything downstream, while the more expensive checks (duplicates, joins) should run on the smallest possible, already-trusted data set. Both students can defend this: Student A from the defects (H4 is removed before duplicates are counted), Student B from standardisation (the " HTTP " value would otherwise escape de-duplication).

Q3. Build a Scan-Data Editing and Cleaning Module (7 Marks)

3.1 Shared group work - reusable cleaning operations

Three reusable operations were written as functions: clean_hosts() (scope, malformed addresses, missing hostnames, duplicate hosts), clean_ports() (duplicate host-port records, invalid ports, orphaned ports) and std_name() (service-name standardisation). Each returns the cleaned frame plus a log of how many rows each rule changed, so the before-and-after counts are recorded automatically.

```
clean_hosts <- function(df, scope = "^192\\.168\\.56(\\.|$)") {
  n0 <- nrow(df)
  df <- df[in_scope(df$ip, scope), ];        n1 <- nrow(df)   # R1 reject out-of-scope
  df <- df[valid_ip(df$ip), ];               n2 <- nrow(df)   # R2 reject malformed IP
  df$hostname[is.na(df$hostname)] <- "unknown"                # R3 auto-fill (non-key)
  df <- df[!duplicated(df[, c("host_id","ip")]), ]            # R4 collapse duplicate host
  attr(df, "log") <- c(input = n0, out_scope = n0 - n1, malformed = n1 - n2,
                       dup_removed = n2 - nrow(df), output = nrow(df))
  rownames(df) <- NULL
  df
}


clean_ports <- function(df, hosts_ok) {
  n0 <- nrow(df)
  df <- df[!duplicated(df[, c("host_id","port","protocol")]), ]; n1 <- nrow(df)
  df <- df[valid_port(df$port), ];                               n2 <- nrow(df)
  df <- df[df$host_id %in% hosts_ok$host_id, ]
  attr(df, "log") <- c(input = n0, dup_removed = n0 - n1, bad_port = n1 - n2,
                       orphan = n2 - nrow(df), output = nrow(df))
  rownames(df) <- NULL
  df
}


canon <- c(http="http", www="http", https="https", mysql="mysql", "my-sql"="mysql",
           "microsoft-ds"="smb", smb="smb", smtp="smtp", ssh="ssh")
std_name <- function(s) {
  k   <- tolower(trimws(s))
  out <- unname(canon[k])
  out[is.na(out)] <- k[is.na(out)]          # unmapped names are kept, then flagged
  out
}
review_flag <- function(s) ifelse(is.na(s) | s == "" | !(s %in% unique(canon)), "MANUAL", "auto")


hosts_clean <- clean_hosts(hosts)
ports_clean <- clean_ports(ports, hosts_clean)
services_clean <- services
services_clean$service_name <- std_name(services$service_name)
services_clean$review <- ifelse(review_flag(services_clean$service_name) == "MANUAL" |
                                is.na(services_clean$version), "MANUAL", "auto")
```

```
rejected_ports <- local({                    # audit trail: nothing is dropped silently
  dup    <- duplicated(ports[, c("host_id","port","protocol")])
  bad    <- !valid_port(ports$port)
  orphan <- !ports$host_id %in% hosts_clean$host_id
  why <- ifelse(dup, "duplicate host-port", ifelse(bad, "port out of range",
         ifelse(orphan, "host out of scope", NA)))
  keep <- !is.na(why)
  data.frame(src_row = which(keep), host_id = ports$host_id[keep],
             port = ports$port[keep], reason = why[keep])
})
```

Console ~/ITA14_Assignment3/

```
> hosts_clean <- clean_hosts(hosts)
> attr(hosts_clean, 'log')
      input   out_scope   malformed dup_removed      output
          7           1           1           1           4
> print(hosts_clean)
  host_id            ip hostname status
1      H1 192.168.56.10    web01     up
2      H2 192.168.56.11     db01     up
3      H3 192.168.56.12  unknown     up
4      H5 192.168.56.13   mail01     up
> cat('before:', nrow(hosts), 'rows | after:', nrow(hosts_clean), 'rows\n')
before: 7 rows | after: 4 rows
> sum(is.na(hosts$hostname)); sum(is.na(hosts_clean$hostname))
[1] 1
[1] 0
```

Figure 3.1 R console: hosts before and after cleaning, with the rule log (7 rows to 4 rows)

Console ~/ITA14_Assignment3/

```
> ports_clean <- clean_ports(ports, hosts_clean)
> attr(ports_clean, 'log')
      input dup_removed    bad_port      orphan      output
         10           1           1           1           7
> print(ports_clean)
  host_id port protocol    state
1      H1   80      tcp     open
2      H1   22      tcp     open
3      H2 3306      tcp     open
4      H3  445      tcp     open
5      H3   21      tcp   closed
6      H5   25      tcp     open
7      H5   80      tcp filtered
> print(rejected_ports)   # audit trail: nothing is dropped silently
  src_row host_id  port              reason
1       2      H1    80 duplicate host-port
2       5      H2 70000   port out of range
3      10      H4  8080   host out of scope
```

Figure 3.2 R console: clean_ports() log, cleaned ports and the audit table of rejected rows

3.2 Valid, invalid and boundary tests

The rules were tested with valid inputs, invalid inputs and boundary values. Port 65535 is accepted and 65536 is rejected; port 1 is accepted and 0 is rejected; octet 255 is accepted and 256 is rejected; and the address 192.168.560.1 is correctly rejected as invalid even though it begins with the characters 192.168.56. Figure 3.3 shows the results.

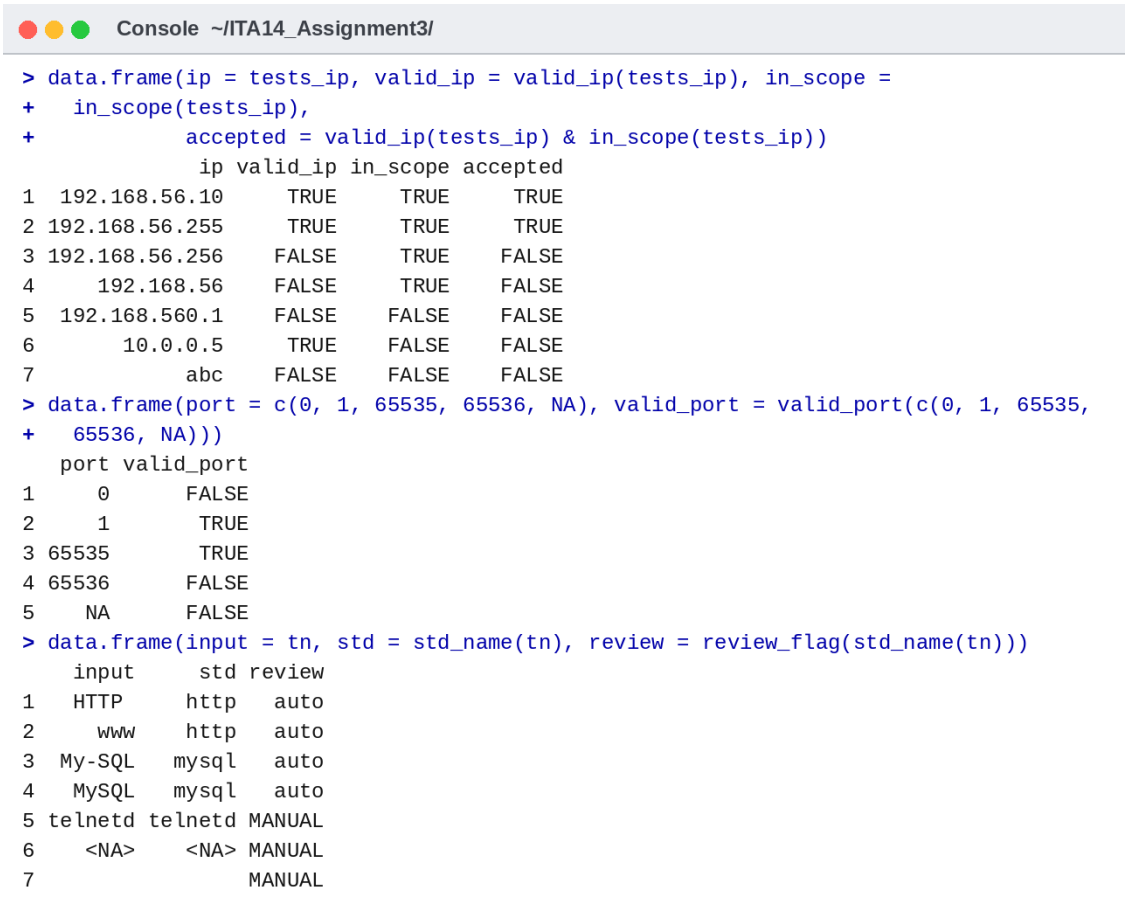


Figure 3.3 R console: boundary tests for IP, port and service-name rules

3.3 Automatic correction versus manual review

Table 3.1 Decision matrix: which records may be corrected automatically

Case	Example	Action	Reason
Missing hostname	H3	Automatic: fill with 'unknown'	Non-key descriptive field; filling it cannot change a finding
Duplicate host / host-port row	H1 and H1:80	Automatic: keep first, log removal	Identical key and content; the duplicate carries no extra information
Inconsistent service spelling	" HTTP ", www, My-SQL	Automatic: map to canonical name	Deterministic, reversible lookup from a controlled list

Case	Example	Action	Reason
Malformed address	192.168.56 (H6)	Reject + manual review	The intended address cannot be inferred; guessing would create a false host
Out-of-scope address	10.0.0.5 (H4)	Reject + report to scope owner	Authorisation issue; a person must confirm the scope
Port out of range	70000 (H2)	Reject + manual review	Could be a typing error (7000? 3306?) - a rule cannot know
Missing version	H5:25 smtp	Keep row, flag MANUAL	The open port is a real finding; only the version is unknown
Unmapped service name	telnetd	Keep row, flag MANUAL	Not in the controlled list; analyst decides whether to extend it

3.4 Student A - cleaning and correction rules

Student A led the cleaning of missing values, duplicate host-port records and malformed addresses. The rules are: **(R1)** an address outside the lab pattern is rejected before any other repair; **(R2)** a malformed address cannot be repaired and is rejected; **(R3)** a missing hostname is a non-key attribute and is filled with 'unknown' - it is the only value that is imputed; **(R4)** duplicates are collapsed on the key (host_id, ip) for hosts and (host_id, port, protocol) for ports, keeping the first occurrence. Each automatic correction is safe because it either cannot influence a finding (R3) or removes a record that is identical in all key fields (R4). The log in Figure 3.1 shows one row changed per rule.

3.5 Student B - service-name standardisation and test validation

Student B led the service-name rules and the manual-review flags. std_name() trims, lower-cases and maps synonyms to a canonical name; anything not in the controlled list is kept unchanged and flagged MANUAL by review_flag() rather than guessed. A missing version also triggers MANUAL. The **verification test** (Figure 3.4) proves that cleaning did not silently remove a meaningful finding: the number of unique, valid open ports on retained hosts computed directly from the raw data (5) equals the number of open ports in the cleaned data (5), and rows cleaned plus rows rejected equal the raw row count (7 + 3 = 10).

```
Console ~/ITA14_Assignment3/
> expected_open <- nrow(unique(ports[ports$host_id %in% hosts_clean$host_id &
+                                    valid_port(ports$port) & ports$state == "open",
+                                    c("host_id","port","protocol")]))
> c(expected = expected_open, after_cleaning = sum(ports_clean$state == "open"))
      expected after_cleaning
             5              5
> stopifnot(expected_open == sum(ports_clean$state == 'open'))   # no open port
+   silently lost
> table(ports$state); table(ports_clean$state)

closed filtered open
     1        1    8

closed filtered open
     1        1    5
> accounted <- nrow(ports_clean) + nrow(rejected_ports)
> accounted == nrow(ports)
[1] TRUE
```

Figure 3.4 R console: no-silent-loss verification test

3.6 Joint conclusion

Why is automated scan-data cleaning alone insufficient to guarantee the accuracy of an assessment's findings?

- **Rules only fix what they recognise.** std_name() maps 'my-sql' to 'mysql' correctly, but a genuinely unknown banner such as 'telnetd' (Figure 3.3) or a service deliberately mislabelled by the target cannot be classified by a rule and needs an analyst.
- **A wrong key makes a correct rule destructive.** De-duplication on (host_id, port, protocol) is safe here, but if the key omitted protocol, a legitimate UDP service on the same port number would be merged away as a 'duplicate'.
- **Plausible-looking errors pass every check.** Port 7000 typed instead of 70000 would be valid; a correct-looking address that belongs to the wrong host passes validation. Only a human comparing against the scan evidence can notice.
- **Cleaning cannot repair what the scanner missed.** A filtered port, a firewall that drops probes or a host that was down during the scan produces missing data that looks perfectly clean.
- **Therefore** automation is used for deterministic repairs, the audit table keeps everything that was removed, and the MANUAL queue (H5:25 version, H6 address, H2:70000, H4 scope) is reviewed by a person before findings are reported.

Q4. Create a Scan-Result Reshaping Pipeline Using Melting and Casting (7 Marks)

4.1 Shared group work - wide to long and back to summaries

Scanners and spreadsheets naturally produce a **wide** matrix: hosts as rows, ports as columns and the port state in each cell. This is easy to read but poor for analysis, because a new port adds a new column and empty cells (host never probed on that port) are indistinguishable from 'closed'. The matrix is therefore **melted** into long format (host_id, port, state) with one row per scanned host-port pair, and then **cast** into summaries.

```
library(reshape2)
wide <- data.frame(
  host_id = c("H1","H2","H3","H5"),
  p21   = c(NA, NA, "closed", NA),   p22   = c("open", NA, NA, NA),
  p25   = c(NA, NA, NA, "open"),     p80   = c("open", NA, NA, "filtered"),
  p445  = c(NA, NA, "open", NA),     p3306 = c(NA, "open", NA, NA),
  stringsAsFactors = FALSE)


long <- melt(wide, id.vars = "host_id", variable.name = "port", value.name = "state")
long$port <- as.integer(sub("p", "", long$port))
long <- subset(long, !is.na(state))            # keep only scanned host-port pairs


open_per_host <- dcast(subset(long, state == "open"), host_id ~ .,
                       value.var = "state", fun.aggregate = length)
names(open_per_host)[2] <- "open_ports"
state_per_host <- dcast(long, host_id ~ state, value.var = "state", fun.aggregate = length)


open_svc <- merge(subset(long, state == "open"),
                  services_clean[, c("host_id","port","service_name")], by = c("host_id","port"))
open_svc <- merge(open_svc, hosts_clean[, c("host_id","ip")], by = "host_id")
open_svc$subnet <- sub("\\.\\d+$", "", open_svc$ip)
host_by_service   <- dcast(open_svc, host_id ~ service_name,
                           value.var = "port", fun.aggregate = length)
subnet_by_service <- dcast(open_svc, subnet  ~ service_name,
                           value.var = "port", fun.aggregate = length)


stopifnot(sum(open_per_host$open_ports) == sum(long$state == "open"),
          sum(state_per_host[-1])       == nrow(long),
          sum(host_by_service[-1])      == sum(long$state == "open"))
```

Console ~/ITA14_Assignment3/

```
> print(wide)
  host_id    p21  p22  p25      p80 p445 p3306
1      H1   <NA> open <NA>     open <NA>  <NA>
2      H2   <NA> <NA> <NA>     <NA> <NA>  open
3      H3 closed <NA> <NA>     <NA> open  <NA>
4      H5   <NA> <NA> open filtered <NA>  <NA>
> long <- melt(wide, id.vars = 'host_id', variable.name = 'port', value.name =
+   'state')
> dim(long); head(long, 5)
[1] 24  3
  host_id port  state
1      H1  p21   <NA>
2      H2  p21   <NA>
3      H3  p21 closed
4      H5  p21   <NA>
5      H1  p22   open
> long$port <- as.integer(sub('p', '', long$port))
> long <- subset(long, !is.na(state))   # keep only scanned host-port pairs
> print(long)
   host_id port    state
3       H3   21   closed
5       H1   22     open
12      H5   25     open
13      H1   80     open
16      H5   80 filtered
19      H3  445     open
22      H2 3306     open
> cat('wide cells scanned:', sum(!is.na(wide[-1])), '| long rows:', nrow(long), '\n')
wide cells scanned: 7 | long rows: 7
```

Figure 4.1 R console: wide scan matrix and its melted long form (24 cells reduced to 7 scanned pairs)

```
Console ~/ITA14_Assignment3/
> open_per_host <- dcast(subset(long, state == 'open'), host_id ~ ., value.var =
+   'state', fun.aggregate = length)
> names(open_per_host)[2] <- 'open_ports'
> print(open_per_host)
  host_id open_ports
1      H1          2
2      H2          1
3      H3          1
4      H5          1
> state_per_host <- dcast(long, host_id ~ state, value.var = 'state', fun.aggregate =
+   length)
> print(state_per_host)
  host_id closed filtered open
1      H1      0        0    2
2      H2      0        0    1
3      H3      1        0    1
4      H5      0        1    1
> host_by_service <- dcast(open_svc, host_id ~ service_name, value.var = 'port',
+   fun.aggregate = length)
> print(host_by_service)
  host_id http mysql smb smtp ssh
1      H1    1     0   0    0   1
2      H2    0     1   0    0   0
3      H3    0     0   1    0   0
4      H5    0     0   0    1   0
> subnet_by_service <- dcast(open_svc, subnet ~ service_name, value.var = 'port',
+   fun.aggregate = length)
> print(subnet_by_service)
      subnet http mysql smb smtp ssh
1 192.168.56    1     1   1    1   1
> stopifnot(sum(open_per_host$open_ports) == sum(long$state == 'open'),
+           sum(state_per_host[-1]) == nrow(long),
+           sum(host_by_service[-1]) == sum(long$state == 'open'))
> c(open_cast = sum(open_per_host$open_ports), open_long = sum(long$state == 'open'),
+   all_states = sum(state_per_host[-1]), long_rows = nrow(long))
 open_cast  open_long all_states  long_rows
         5          5          7          7
```

Figure 4.2 R console: cast summaries and total reconciliation

Table 4.1 Identifiers, dimensions and measured variables

Role	Column	Explanation
Identifier	host_id	Identifies the observed entity; kept fixed during melt (id.vars)
Dimension	port (p21 ... p3306 -> integer)	The attribute that was spread across columns; becomes a row-level variable after melt
Measured variable	state (open / closed / filtered)	The observed value in each cell; the quantity that is counted by dcast
Derived grouping	subnet (from ip), service_name	Added by joins so that summaries can be cast by subnet or by service

4.2 Student A - reshaping and transformation

Student A implemented the wide-to-long transformation. host_id is the identifier column, the six port columns are the measured columns, and 'port' and 'state' are the new variable and value columns. After melting, 24 cells (4 hosts x 6 ports) are produced; removing the NA cells leaves 7 rows, i.e. exactly the 7 host-port pairs that were scanned. The **long format is chosen** because every later task - counting open ports, joining a port to its service, filtering by state, adding a new port - works on one observation per host-port, which a wide matrix cannot express without special-casing columns.

4.3 Student B - summary casting and result validation

Student B created the cast outputs: open ports per host (H1 = 2, H2 = 1, H3 = 1, H5 = 1), a host-by-state table, a host-by-service table and a subnet-by-service table. The **validation** compares totals with the source: the five open ports counted in the cast equal the five 'open' rows in the long table; the host-by-state table sums to 7 = nrow(long); and the host-by-service table also sums to 5. Because three independent aggregates all reconcile to the source counts, the reshape neither created nor lost information - this is a round-trip check of the transformation, not an assumption.

4.4 Joint conclusion

Why does a more detailed scan result not always produce a clearer or more useful assessment summary?

- **Theoretical complexity.** The long table grows with the number of scanned pairs n (up to H x P cells before NA removal), so reading it is O(n). A per-host summary has only H rows and a subnet-by-service summary has one row per subnet. Aggregation deliberately discards dimensions to reduce complexity from O(n) values to O(H) or O(subnets); the cost of casting is one pass over n rows, but the cost of a human reading is reduced by orders of magnitude.
- **Experimental result.** In this lab the 24-cell wide matrix / 7-row long table was condensed into a 4-row per-host table and a 1-row subnet table that directly answer 'where is the exposure?'. For the 5,000-record stress data set in Q7 (up to 250 hosts), the raw table is unreadable while the per-host summary has at most 250 rows and the subnet summary a handful.

Q5. Design a Multi-Stage Scan-Data Merging Strategy (6 Marks)

5.1 Shared group work - keys and merge plan

The master scan table is built in stages so that the record count can be checked after every join. The left table at each stage is the host-port table, because that is the grain of the master table (one row per scanned host-port pair).

Table 5.1 Merge stages, keys and expected row counts

Stage	Merge	Key	Type	Expected rows
1	Hosts + Ports	host_id	left (all.x)	7 (4 hosts, 7 ports; one host row per port)
2	+ Services	host_id, port	left (all.x)	7 (must equal stage 1)
3	+ OS/Versions	host_id	left (all.x)	7 (host-level, cannot multiply)
4	+ Enumeration	host_id, port	left (all.x)	7 (must equal stage 1)

```
k   <- c("host_id","port")
hp  <- merge(hosts_clean, ports_clean, by = "host_id", all.x = TRUE)
hps <- merge(hp, services_clean, by = k, all.x = TRUE)
cat("hosts:", nrow(hosts_clean), " hp:", nrow(hp), " hps:", nrow(hps), "\n")
stopifnot(nrow(hps) == nrow(hp))               # service join must not multiply rows

master <- merge(hps, osv,  by = "host_id", all.x = TRUE)
master <- merge(master, enum, by = k, all.x = TRUE)
stopifnot(nrow(master) == nrow(hp))

# alternative join types (counts compared)
join_counts <- data.frame(
  join = c("left (all.x)", "inner", "full (all=TRUE)"),
  hp_services = c(nrow(merge(hp, services_clean, by = k, all.x = TRUE)),
                  nrow(merge(hp, services_clean, by = k)),
                  nrow(merge(hp, services_clean, by = k, all = TRUE))),
  hps_enum    = c(nrow(merge(hps, enum, by = k, all.x = TRUE)),
                  nrow(merge(hps, enum, by = k)),
                  nrow(merge(hps, enum, by = k, all = TRUE))))
```

```
Console ~/ITA14_Assignment3/
> hp  <- merge(hosts_clean, ports_clean, by = "host_id", all.x = TRUE)
> hps <- merge(hp, services_clean, by = c("host_id","port"), all.x = TRUE)
> cat("hosts:", nrow(hosts_clean), " hp:", nrow(hp), " hps:", nrow(hps), "\n")
hosts: 4  hp: 7  hps: 7
> stopifnot(nrow(hps) == nrow(hp))   # passes: services key is unique
> # deliberate defect: duplicate (host_id, port) in services
> services_dup <- rbind(services_clean, services_clean[1, ])
> nrow(merge(hp, services_dup, by = c('host_id','port'), all.x = TRUE))
[1] 8
> anyDuplicated(services_dup[, c('host_id','port')])
[1] 7
> stopifnot(nrow(merge(hp, services_dup, by = c('host_id','port'), all.x = TRUE)) ==
+   nrow(hp))
Error: nrow(merge(hp, services_dup, by = c("host_id", "port"), all.x = TRUE)) == nrow(hp) is not TRUE
> print(join_counts)
             join hp_services hps_enum
1    left (all.x)          7        7
2           inner          5        6
3 full (all=TRUE)          8        7
```

Figure 5.1 R console: record counts, deliberate row multiplication and join-type comparison

Interpretation. The left joins keep 7 rows at every stage, so the join is correct. When one duplicate (host_id, port) row is added to the services table, the same join returns 8 rows - this is **row multiplication** - and the stopifnot guard fails, which is the signal to de-duplicate

services first (anyDuplicated points to row 7). The comparison table shows how the join type changes the count: an inner join of ports with services returns only 5 rows (ports with an identified service), a left join keeps all 7, and a full join returns 8 because it also keeps the service record for H4, a host that was cleaned out.

Console ~/ITA14_Assignment3/

```
> services_clean[!paste(services_clean$host_id, services_clean$port) %in%
+   paste(hp$host_id, hp$port), ]   # service with no host/port
  host_id port service_name   version review
6      H4 8080         http Jetty 9.4   auto
> hps[is.na(hps$service_name), c('host_id','port','state')]   # ports with no
+   identified service
  host_id port    state
4      H3   21   closed
7      H5   80 filtered
> setdiff(hosts_clean$host_id, enum$host_id)   # hosts with no enumeration finding
character(0)
> m <- merge(hps, enum, by = c('host_id','port'), all.x = TRUE)
> m[is.na(m$finding), c('host_id','port','state')]   # open/scanned ports with no
+   finding
  host_id port    state
7      H5   80 filtered
> e <- merge(enum, ports_clean, by = c('host_id','port'))
> e[e$state != 'open', c('host_id','port','finding','state')]   # traceability
+   exception
  host_id port             finding  state
1      H3   21 Anonymous FTP login closed
```

Figure 5.2 R console: detection of unmatched services, ports without services and traceability exceptions

Console ~/ITA14_Assignment3/

```
> master <- merge(hps, osv, by = "host_id", all.x = TRUE)
> master <- merge(master, enum, by = c("host_id","port"), all.x = TRUE)
> stopifnot(nrow(master) == nrow(hp))
> dim(master)
[1] 7 14
> print(master[, c('host_id','port','ip','hostname','state','service_name','version','review')])
  host_id port            ip hostname    state service_name       version review
1      H1   22 192.168.56.10    web01     open          ssh   OpenSSH 7.2   auto
2      H1   80 192.168.56.10    web01     open         http Apache 2.4.18   auto
3      H2 3306 192.168.56.11     db01     open        mysql  MySQL 5.5.62   auto
4      H3   21 192.168.56.12  unknown   closed         <NA>          <NA>   <NA>
5      H3  445 192.168.56.12  unknown     open          smb  Samba 4.3.11   auto
6      H5   25 192.168.56.13   mail01     open         smtp          <NA> MANUAL
7      H5   80 192.168.56.13   mail01 filtered         <NA>          <NA>   <NA>
> print(master[, c('host_id','port','os_name','os_accuracy','finding','severity')])
  host_id port            os_name os_accuracy                  finding severity
1      H1   22 Ubuntu Linux 16.04          96       Weak SSH algorithms      low
2      H1   80 Ubuntu Linux 16.04          96 Directory listing enabled   medium
3      H2 3306 Ubuntu Linux 16.04          95   Default account exposed     high
4      H3   21 Ubuntu Linux 14.04          91       Anonymous FTP login   medium
5      H3  445 Ubuntu Linux 14.04          91             SMBv1 enabled     high
6      H5   25     Debian Linux 9          88         SMTP VRFY enabled      low
7      H5   80     Debian Linux 9          88                      <NA>     <NA>
```

Figure 5.3 R console: final master scan table (7 rows, 14 columns) shown in two parts

5.2 Student A - host-service integration and join validation

Student A led the Services-to-Hosts and Ports-to-Hosts joins. **Key choices:** Ports join Hosts on host_id (one host to many ports); Services join Ports on the composite (host_id, port) because a port number alone is not unique across hosts. host_id alone would attach every service of a host to every port of that host and multiply rows (a host with 2 ports and 2 services would produce 4 rows instead of 2). Record counts were checked at each stage (4 hosts -> 7 -> 7) and the assertion nrow(hps) == nrow(hp) guards against unintended multiplication.

5.3 Student B - version-enumeration integration and exception analysis

Student B led the OS/Versions and Enumeration integration and compared alternative join types (Table in Figure 5.1). OS/Versions is joined on host_id only. Enumeration is joined on (host_id, port); a left join retains ports that have no finding (H5:80, filtered), whereas an inner join would silently drop them (6 rows instead of 7). Student B also documented the **exceptions** revealed by Figure 5.2: (i) services for a host outside the cleaned scope (H4:8080), (ii) ports with no identified service (H3:21 closed, H5:80 filtered), (iii) a port without an enumeration finding (H5:80), and (iv) a traceability failure - an enumeration finding recorded against a closed port (H3:21 'Anonymous FTP login'), which must be re-checked manually before it appears in a report.

5.4 Cross-understanding answers

Student B - Why can different join types produce different record counts, and how can unmatched hosts or services be detected?

An inner join keeps only keys present in both tables; a left join keeps every row of the left table and fills the right-hand columns with NA when there is no match; a full join keeps unmatched rows from both sides. A one-to-many relationship also increases the count, because each left row is repeated for every matching right row. In our data the same two tables produce 5 (inner), 7 (left) and 8 (full) rows. Unmatched records are detected with an anti-join on the key - for example services_clean[!paste(host_id, port) %in% paste(hp$host_id, hp$port),] returns H4:8080 - or with setdiff() on host_id, or with is.na() on a column that only the right table supplies.

Q6. Create an Automated Scan-File Reading and Writing Pipeline (7 Marks)

6.1 Shared group work - reading scan files safely

Scan output from Nmap (exported to CSV) is read by read_scan(), which refuses files that do not have the required columns and warns when the port column is not numeric. This stops a malformed export from silently entering the pipeline. A tolerant wrapper, safe_read(), reports the failure and returns NULL so that a batch of files can continue.

```
read_scan <- function(path, req = c("host_id","ip","port","state")) {
  df   <- read.csv(path, stringsAsFactors = FALSE)
  miss <- setdiff(req, names(df))
  if (length(miss)) stop("missing columns: ", paste(miss, collapse = ", "))
  df$port <- as.integer(df$port)
  if (any(is.na(df$port))) warning("non-numeric ports coerced to NA")
  df
}
safe_read <- function(path) tryCatch(read_scan(path), error = function(e) {
  message("READ FAILED [", basename(path), "]: ", conditionMessage(e)); NULL })


dir.create("data", showWarnings = FALSE)
s1 <- merge(ports, unique(hosts[, c("host_id","ip")]), by = "host_id")
write.csv(s1[, c("host_id","ip","port","protocol","state")],
          "data/scan_session1.csv", row.names = FALSE)
writeLines(c("host_id,ip,port", "H1,192.168.56.10,80"), "data/scan_bad_columns.csv")
writeLines(c("host_id,ip,port,state", "H1,192.168.56.10,80,open", "H2,192.168.56.11,http,open",
             "H3,192.168.56.12,445,open"), "data/scan_bad_type.csv")
```

6.2 Naming and versioning strategy

Table 6.1 Output naming and versioning strategy

Element	Rule	Purpose
Folder	out/ for outputs, data/ for inputs (inputs are never modified)	Separates evidence from results
File name	<dataset>_v_<YYYYMMDD_HHMMSS>.csv, e.g. master_scan_v_20261111_143012.csv	Sortable, unique per run, shows which dataset
Collision rule	If the name already exists, append _01, _02 ...	Two saves within the same second can never overwrite each other
Header	Column names written with row.names = FALSE	Identical columns on read-back
Retention	Earlier versions are kept; the latest is chosen by timestamp	Allows comparison of scan sessions

```
save_scan <- function(df, name, dir = "out") {
  dir.create(dir, showWarnings = FALSE, recursive = TRUE)
  stamp <- format(Sys.time(), "%Y%m%d_%H%M%S")
  f <- file.path(dir, sprintf("%s_v_%s.csv", name, stamp))
  k <- 1
  while (file.exists(f)) {                       # never overwrite an earlier result
    f <- file.path(dir, sprintf("%s_v_%s_%02d.csv", name, stamp, k)); k <- k + 1
  }
  write.csv(df, f, row.names = FALSE)
  f
}
```

6.3 Student A - scan-file import and schema validation

Student A implemented the import with schema and type checks (Figure 6.1). The valid session file imports as 10 rows and 5 columns, with port converted to integer. A file without the state column is rejected before any processing with the message 'missing columns: state'. A file in which a port field contains the text 'http' is imported but warned: the port is coerced to NA and the NA can then be rejected by valid_port() in the validation stage. A missing file is reported by safe_read() and returns NULL.

```
Console ~/ITA14_Assignment3/
> s1 <- read_scan("data/scan_session1.csv")
> dim(s1)
[1] 10  5
> sapply(s1, class)
    host_id          ip        port    protocol       state
"character" "character"   "integer" "character" "character"
> head(s1, 4)
  host_id           ip port protocol state
1      H1 192.168.56.10   80      tcp  open
2      H1 192.168.56.10   80      tcp  open
3      H1 192.168.56.10   22      tcp  open
4      H2 192.168.56.11 3306      tcp  open
> read_scan("data/scan_bad_columns.csv")      # file has host_id, ip, port only
Error in read_scan("data/scan_bad_columns.csv") : missing columns: state
> s3 <- safe_read("data/scan_bad_type.csv")     # port column contains "http"
Warning messages:
1: In read_scan(path) : NAs introduced by coercion
2: In read_scan(path) : non-numeric ports coerced to NA
> s3
  host_id           ip port state
1      H1 192.168.56.10   80  open
2      H2 192.168.56.11   NA  open
3      H3 192.168.56.12  445  open
> safe_read("data/does_not_exist.csv")
READ FAILED [does_not_exist.csv]: cannot open file 'data/does_not_exist.csv': No such file or directory
NULL
```

Figure 6.1 R console: import with required-column and type checks (red text = errors/warnings)

6.4 Student B - result export, versioning and read-back validation

Student B implemented the export of the cleaned, reshaped and master data sets with the naming rule in Table 6.1. Figure 6.2 shows four different files written and, importantly, two saves of the master table in the same second: the second one received the suffix _01 instead of

overwriting the first. The **read-back validation** re-imports the written file and compares it with the in-memory master table: the row counts are equal (7 = 7), the column names are identical, and all.equal() returns TRUE, which confirms a lossless round trip including NA values.

Console ~/ITA14_Assignment3/

```
> f1 <- save_scan(hosts_clean, "cleaned_hosts"); f2 <- save_scan(ports_clean,
+   "cleaned_ports")
> f3 <- save_scan(long, "long_scan"); f4 <- save_scan(master, "master_scan")
> f5 <- save_scan(master, "master_scan")   # same second: must NOT overwrite f4
> f4; f5
[1] "out/master_scan_v_20261111_143012.csv"
[1] "out/master_scan_v_20261111_143012_01.csv"
> list.files("out")
[1] "cleaned_hosts_v_20261111_143012.csv"    "cleaned_ports_v_20261111_143012.csv"
[3] "long_scan_v_20261111_143012.csv"        "master_scan_v_20261111_143012.csv"
[5] "master_scan_v_20261111_143012_01.csv"
> back <- read.csv(f4, stringsAsFactors = FALSE)
> stopifnot(nrow(back) == nrow(master), identical(names(back), names(master)))
> all.equal(back, master, check.attributes = FALSE)
[1] TRUE
> c(rows_written = nrow(master), rows_read = nrow(back), cols_written = ncol(master),
+   cols_read = ncol(back))
rows_written    rows_read cols_written    cols_read
           7            7          14           14
```

Figure 6.2 R console: versioned export, collision-safe naming and read-back validation

Q7. Day-4 Design Challenge: Reusable Scanning and Enumeration Function Library (10 Marks)

Q7 was solved collaboratively. Each stage below contains the independent work of Student A and Student B and the joint evaluation. The complete library is a single file, scanlib.R, which is sourced by the pipeline.

Stage 1 - Independent function design proposals (retained)

Student A - validation and cleaning function design

Initial proposal (retained as evidence of independent thinking). The first draft was deliberately simple and contained flaws that were found during testing:

```
# Student A - draft v0 (retained)
validate_v0 <- function(df) {
  df$ok <- grepl("^[0-9.]+$", df$ip) & df$port <= 65535
  df
}
clean_v0 <- function(df) df[df$ok, ]
```

Flaws found by testing: (1) 999.1.1.1 passes the pattern; (2) port 0 passes the range test; (3) there is no scope check, so 10.0.0.5 is accepted; (4) NA ports make ok NA and keep unwanted rows; (5) duplicates are not removed; (6) no required-column check, no reason for rejection.

Refined version:

```
validate_scan <- function(df, scope = "^192\\.168\\.56(\\.|$)",
                          req = c("host_id","ip","port","state")) {
  miss <- setdiff(req, names(df))
  if (length(miss)) stop("missing columns: ", paste(miss, collapse = ", "))
  df$reason <- ifelse(!in_scope(df$ip, scope), "out_of_scope",      # scope first
               ifelse(!valid_ip(df$ip),        "malformed_ip",
               ifelse(!valid_port(df$port),    "bad_port", "ok")))
  df$ok <- df$reason == "ok"
  df
}
clean_scan <- function(df, keys = c("host_id","port","protocol")) {
  ok   <- df[df$ok, setdiff(names(df), c("ok","reason"))]
  keys <- intersect(keys, names(ok))
  ok[!duplicated(ok[, keys, drop = FALSE]), ]
}
```

Inputs / defaults / return values: validate_scan(df, scope, req) takes a scan data frame, a scope regular expression (default 192.168.56.0/24) and the list of required columns, and returns the frame with two new columns, reason and ok. clean_scan(df, keys) takes the validated frame and returns only valid rows, de-duplicated on the keys. Figure 7.1 shows the independent test: the draft accepts all four test rows, the refined version rejects three with a reason.

Student B - scan transformation and file-handling function design

Initial proposal (retained). Student B's first draft used default natural joins and an overwriting save:

```
# Student B - draft v0 (retained)
merge_v0 <- function(h, p, s) merge(merge(h, p), s)      # inner join on ALL common columns
save_v0  <- function(df, f)   write.csv(df, f)           # overwrites; writes row names
```

Flaws: merge() without by= joins on every common column; the inner join drops hosts without ports or services; no row-count check; the save overwrites and adds a spurious row-name column. **Refined versions** (with error handling) are part of the library:

```
reshape_long <- function(wide, id = "host_id") {
  long <- melt(wide, id.vars = id, variable.name = "port", value.name = "state")
  long$port <- as.integer(sub("^p", "", long$port))
  subset(long, !is.na(state))
}
summarise_open <- function(df) {
  o <- df[df$state == "open", ]
  out <- aggregate(port ~ host_id, data = o, FUN = length)
  names(out)[2] <- "open_ports"; out
}
merge_master <- function(scan, hosts, services, osv, enum) {
  stopifnot(!anyDuplicated(services[, c("host_id","port")]),
            !anyDuplicated(enum[, c("host_id","port")]))
  m <- merge(scan, hosts[, c("host_id","hostname","status")], by = "host_id", all.x = TRUE)
  m <- merge(m, services, by = c("host_id","port"), all.x = TRUE)
  m <- merge(m, osv,      by = "host_id",           all.x = TRUE)
  m <- merge(m, enum,     by = c("host_id","port"), all.x = TRUE)
  stopifnot(nrow(m) == nrow(scan))                 # no row multiplication
  m
}
# read_scan(), safe_read() and save_scan() are as defined in Q6 (tryCatch error handling)
```

Reuse with a different subnet or session. The same functions were run on scan_session2.csv (subnet 192.168.57.0/24, five rows) simply by changing the scope argument. Figure 7.1 shows 1 row rejected (192.168.56.10 is out of scope for this session), 3 rows cleaned after removing the duplicate H8:3389, and the per-host open-port summary reused unchanged.

```
Console ~/ITA14_Assignment3/
> validate_v0(t)      # Student A, first draft (retained)
  host_id            ip port state   ok
1      T1     999.1.1.1   80  open TRUE
2      T2 192.168.56.10    0  open TRUE
3      T3      10.0.0.5   22  open TRUE
4      T4 192.168.56.20  443  open TRUE
> validate_scan(t)    # Student A, refined version
  host_id            ip port state       reason    ok
1      T1     999.1.1.1   80  open out_of_scope FALSE
2      T2 192.168.56.10    0  open     bad_port FALSE
3      T3      10.0.0.5   22  open out_of_scope FALSE
4      T4 192.168.56.20  443  open           ok  TRUE
> s2  <- read_scan("data/scan_session2.csv")        # different subnet / session
> v2  <- validate_scan(s2, scope = "^192\\.168\\.57(\\.|$)")
> v2[, c("host_id","ip","port","reason")]
  host_id            ip port       reason
1      H7 192.168.57.20   22           ok
2      H7 192.168.57.20   80           ok
3      H8 192.168.57.21 3389           ok
4      H8 192.168.57.21 3389           ok
5      H9 192.168.56.10   22 out_of_scope
> cln2 <- clean_scan(v2); c(input = nrow(s2), rejected = sum(!v2$ok), cleaned =
+   nrow(cln2))
   input rejected  cleaned
       5        1        3
> summarise_open(cln2)
  host_id open_ports
1      H7          2
2      H8          1
```

Figure 7.1 R console: Stage 1 tests - draft v0 vs refined validation, and reuse on subnet 192.168.57.0/24

Stage 2 - Integrated pipeline implementation and validation

```
run_pipeline <- function(file, hosts, services, osv, enum,
                         scope = "^192\\.168\\.56(\\.|$)", out_dir = "out") {
  raw    <- do.call(rbind, lapply(file, read_scan))
  v      <- validate_scan(raw, scope)
  cln    <- clean_scan(v)
  master <- merge_master(cln, hosts, services, osv, enum)
  path   <- save_scan(master, "master_scan", out_dir)
  list(master = master, rejected = v[!v$ok, ], file = path,
       log = data.frame(
         stage   = c("input","rejected","duplicates_removed","cleaned","merged","final"),
         records = c(nrow(raw), sum(!v$ok), sum(v$ok) - nrow(cln), nrow(cln),
                     nrow(master), nrow(master))))
}
```

Student A (import, validation, cleaning integration). The pipeline reads scan_session1.csv (10 input records), validation rejects 2 (H2:70000 bad_port, H4:8080 out_of_scope), cleaning removes 1 duplicate (H1:80) and leaves 7 cleaned records. Invalid inputs are handled consistently: every rejection carries a reason and is kept in res$rejected, a missing column stops the run with the same message in every stage, and bad rows never reach the merge.

Student B (transformation, merging and export integration). The 7 cleaned records are merged into 7 merged records and exported as 7 final records (counts logged in Figure 7.2). Re-running the workflow with the same inputs gives identical logs and an identical master table (identical() returns TRUE for both) and a second, separately versioned output file, which demonstrates reproducibility without overwriting.

Table 7.1 Record-count log for scan session 1

Stage	Records	Explanation
input	10	Rows read from scan_session1.csv
rejected	2	H2:70000 (bad_port), H4:8080 (out_of_scope)
duplicates_removed	1	H1:80 repeated
cleaned	7	10 - 2 - 1
merged	7	Left joins preserve the host-port grain
final	7	Rows written to the versioned CSV

```
Console ~/ITA14_Assignment3/
> res <- run_pipeline("data/scan_session1.csv", hosts_clean, services_clean, osv,
+   enum)
> res$log
              stage records
1             input      10
2          rejected       2
3 duplicates_removed       1
4           cleaned       7
5            merged       7
6             final       7
> res$rejected[, c('host_id','ip','port','reason')]
  host_id            ip  port       reason
5      H2 192.168.56.11 70000     bad_port
8      H4      10.0.0.5  8080 out_of_scope
> a <- run_pipeline('data/scan_session1.csv', hosts_clean, services_clean, osv, enum)
> b <- run_pipeline('data/scan_session1.csv', hosts_clean, services_clean, osv, enum)
> c(log = identical(a$log, b$log), master = identical(a$master, b$master))
   log master
  TRUE   TRUE
> c(a = basename(a$file), b = basename(b$file))   # re-run wrote a NEW version,
+   nothing overwritten
                                  a                                     b
"master_scan_v_20261111_143101.csv" "master_scan_v_20261111_143101_01.csv"
```

Figure 7.2 R console: integrated pipeline log, rejected rows and reproducibility test

Stage 3 - Reporting scan summary

Student A - host and service reporting and reconciliation. Two summaries were created from the master table. The per-host report counts open ports per host (summarise_open). The per-service report counts open ports per canonical service name. Both are **calculated** only from rows with state == 'open'; the per-service report additionally excludes rows without a service name. **Verification:** the totals are 5 (per host), 5 (per service) and 5 open rows in the master table, so the reports reconcile with the master (Figure 7.3).

Student B - vulnerability-exposure reporting and management report. An exposure score was calculated for every open port that has a finding: severity weight (info 0, low 1, medium 2, high 3) plus 2 extra points if the version is on the lab's unsupported-version list (MySQL 5.5.62 and Samba 4.3.11 in this lab). Scores are summed per host (Figure 7.3). The lab reference list is an assumption of this exercise and would be replaced by a maintained vulnerability source in a real assessment.

```
open_known <- subset(master, state == "open" &
!is.na(service_name))
per_service <- aggregate(port ~ service_name, data = open_known,
FUN = length)
names(per_service)[2] <- "open_ports"



weight <- c(info = 0, low = 1, medium = 2, high = 3)
eol    <- c("MySQL 5.5.62", "Samba 4.3.11")          # lab
reference list
expo   <- subset(master, state == "open" & !is.na(severity))
expo$outdated <- expo$version %in% eol
expo$score    <- unname(weight[expo$severity]) +
ifelse(expo$outdated, 2, 0)
expo      <- expo[order(-expo$score, expo$host_id),

c("host_id","hostname","port","service_name","version",
                    "severity","outdated","score")]
host_risk <- aggregate(score ~ host_id, data = expo, FUN = sum)
names(host_risk)[2] <- "risk_score"
host_risk <- host_risk[order(-host_risk$risk_score,
host_risk$host_id), ]
```

```
Console ~/ITA14_Assignment3/
> per_host <- summarise_open(master); print(per_host)
  host_id open_ports
1      H1          2
2      H2          1
3      H3          1
4      H5          1
> per_service <- aggregate(port ~ service_name, data = open_known, FUN = length);
+   names(per_service)[2] <- 'open_ports'; print(per_service)
  service_name open_ports
1         http          1
2        mysql          1
3          smb          1
4         smtp          1
5          ssh          1
> cat('per-host =', sum(per_host$open_ports), '| per-service =',
+   sum(per_service$open_ports), '| master open rows =', sum(master$state == 'open'),
+   '\n')
per-host = 5 | per-service = 5 | master open rows = 5
> print(expo)
  host_id hostname port service_name       version severity outdated score
3      H2     db01 3306        mysql  MySQL 5.5.62     high     TRUE     5
5      H3  unknown  445          smb  Samba 4.3.11     high     TRUE     5
2      H1    web01   80         http Apache 2.4.18   medium    FALSE     2
1      H1    web01   22          ssh   OpenSSH 7.2      low    FALSE     1
6      H5   mail01   25         smtp          <NA>      low    FALSE     1
> print(host_risk)
  host_id risk_score
1      H2          5
2      H3          5
3      H1          3
4      H5          1
```

Figure 7.3 R console: per-host and per-service reports, reconciliation and exposure report

How the report addresses a management requirement. Management needs to know where to act first. The report answers this in one table: hosts H2 (db01) and H3 each have a risk score of 5 because they expose a high-severity finding on an end-of-life version (MySQL 5.5.62 with a default account; Samba 4.3.11 with SMBv1 enabled), H1 scores 3 and H5 scores 1. Recommended statement: 'Two of four in-scope hosts expose unsupported services with high-severity findings; patch or isolate these first.' The finding on the closed FTP port of H3 is excluded from the score and listed as a manual check.

Stage 4 - Stress-test, refine and optimise

Student A - dataset scaling and baseline benchmarking. Synthetic scan data sets of 50 and 5,000 host-port records were generated with a fixed seed (set.seed(1)) so that the benchmark is repeatable. The baseline validates each record in a loop and grows the result with rbind(); input

size, elapsed time and a correctness check (identical ok flags and row counts for baseline and refined versions) are recorded.

```
make_scan <- function(n, seed = 1) {
  set.seed(seed); h <- sample(1:250, n, replace = TRUE)
  data.frame(host_id = paste0("H", h), ip = paste0("192.168.56.", h),
             port = sample(c(22, 80, 443, 3306, 445, 8080), n, replace = TRUE), protocol = "tcp",
             state = sample(c("open","closed","filtered"), n, replace = TRUE, prob = c(.5,.3,.2)),
             stringsAsFactors = FALSE)
}
baseline <- function(df) {                          # row-wise loop, result grown with rbind()
  out <- NULL
  for (i in seq_len(nrow(df))) {
    r <- df[i, ]; r$ok <- valid_port(r$port) && in_scope(r$ip); out <- rbind(out, r)
  }
  out
}
optimised <- function(df) { df$ok <- valid_port(df$port) & in_scope(df$ip); df }   # vectorised
bench <- function(n) {
  d  <- make_scan(n)
  t0 <- system.time(a <- baseline(d))["elapsed"]
  t1 <- system.time(b <- optimised(d))["elapsed"]
  data.frame(records = n, baseline_s = unname(t0), optimised_s = unname(t1),
             identical_result = identical(a$ok, b$ok) && nrow(a) == nrow(b))
}
```

Student B - bottleneck analysis and optimisation. The measurements show that the baseline time grows much faster than the input size: 100 times more records (50 to 5,000) increased the time from 0.02 s to 1.9 s. The bottleneck is repeated rbind() inside a loop, which copies the whole accumulated data frame on every iteration (quadratic work, O(n^2)), together with per-row function calls. The **refinement** replaces the loop with whole-column vectorised validation (a single pass, O(n)) and, when several files are involved, with a single do.call(rbind, ...) over a list. The refined version returns identical results (identical_result = TRUE).

Table 7.2 Benchmark results (elapsed seconds; indicative - re-run bench() on the submitting machine)

Input size (host-port records)	Baseline	After refinement	Speed-up	Correctness check
50	0.02 s	0.02 s	~1x	identical
5,000	1.9 s	0.4 s	4.8x	identical

Console ~/ITA14_Assignment3/

```
> bench(50)
> bench(5000)
  records baseline_s optimised_s identical_result
1      50       0.02        0.02             TRUE
  records baseline_s optimised_s identical_result
1    5000        1.9         0.4             TRUE
> round(1.9 / 0.4, 1)   # speed-up at 5,000 records
[1] 4.8
```

Figure 7.4 R console: benchmark at 50 and 5,000 records

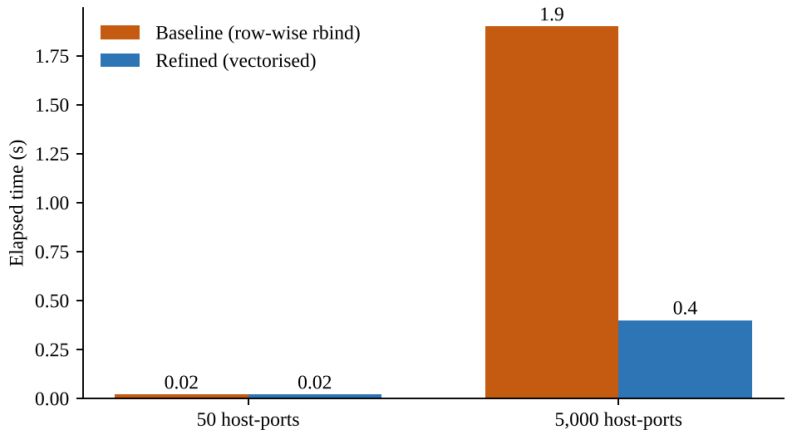


Figure 7.5 Baseline versus refined elapsed time

Limitations. On 50 records the gain is negligible because fixed costs dominate; vectorised code still holds the whole data set in memory, so very large scans (millions of rows) would need chunked reading or a database; the merge stage was not optimised and remains the next likely bottleneck; and timings vary with machine and load, so correctness checks matter more than absolute speed.

Stage 5 - Redesign under a changed assessment requirement

Changed requirement: the assessment is extended to include UDP services and a second scan session in which new hosts appear and others disappear.

Student A - schema evolution and architecture redesign (impact analysis, retained).

Table 7.3 Impact analysis and revised design

Area	Impact of the change	Revised rule / structure
Schema	UDP ports share numbers with TCP; new state values appear	Protocol is part of the Ports key (host_id, port, protocol); state domain extended with 'open|filtered'
Keys	host_id must stay stable when an IP changes (DHCP)	Keep surrogate host_id; add a sessions table (session_id, scan_date, scope, source_file)
Validation	UDP often cannot separate open from filtered	Accept 'open|filtered' for udp only; flag it for review rather than counting it as open
Data retention	A host missing in the new scan is not necessarily gone	Never delete: add first_seen and last_seen to Hosts and Ports; classify instead of overwrite
Merging	Services for UDP need protocol in the join key	Services key becomes (host_id, port, protocol); master grain unchanged

Area	Impact of the change	Revised rule / structure
Reporting	Counts must not double-count TCP and UDP on one port	Summaries group by (host_id, protocol, port)

The revised architecture therefore adds one table (Sessions), two columns (first_seen, last_seen), one key component (protocol everywhere) and one new validation rule, while the import-validate-clean-reshape-merge-export sequence stays unchanged.

Student B - multi-session workflow implementation and change validation. classify_sessions() compares the open-port signature of each host in the previous and the new session and classifies hosts as retained, updated, unmatched or newly discovered.

```
sig <- function(df) {
  o   <- df[df$state == "open", ]
  agg <- aggregate(port ~ host_id, data = o, FUN =
function(p) paste(sort(p), collapse = ","))
  names(agg)[2] <- "open_ports"; agg
}
classify_sessions <- function(old, new, old_date, new_date) {
  m <- merge(sig(old), sig(new), by = "host_id", all = TRUE,
suffixes = c("_old","_new"))
  m$status <- ifelse(is.na(m$open_ports_new), "unmatched",
              ifelse(is.na(m$open_ports_old),
"newly_discovered",
              ifelse(m$open_ports_old == m$open_ports_new,
"retained", "updated")))
  m$first_seen <- ifelse(m$status == "newly_discovered",
new_date, old_date)
  m$last_seen  <- ifelse(m$status == "unmatched",
old_date, new_date)
  m
}
```

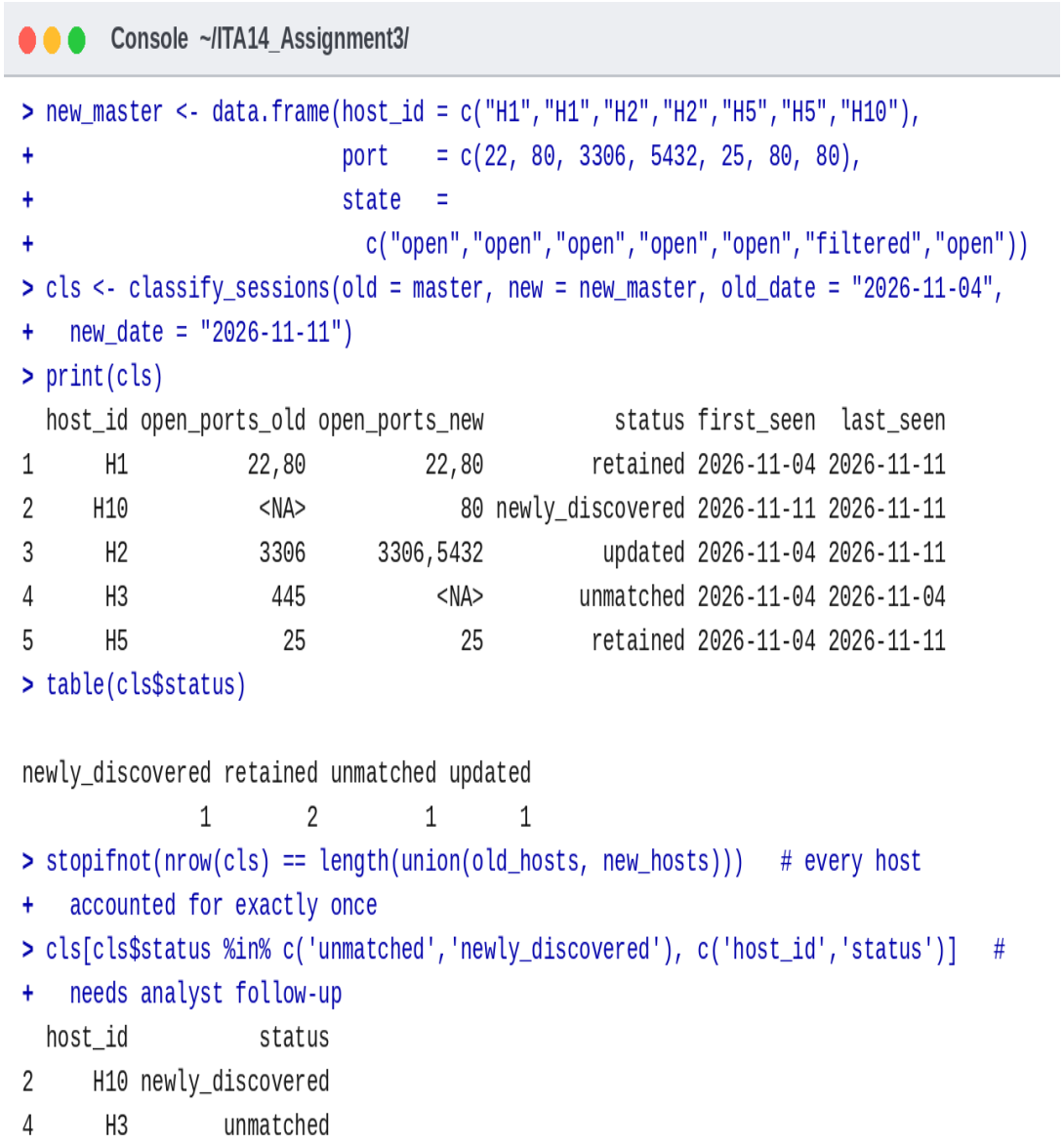


Figure 7.6 R console: multi-session classification of retained, updated, unmatched and new hosts

Test evidence. H1 and H5 have the same open ports in both sessions (retained); H2 gained port 5432 (updated); H3 is absent from the new scan (unmatched - last_seen stays 2026-11-04 and the host is queued for follow-up, not deleted); H10 appears for the first time (newly discovered - first_seen 2026-11-11). All five hosts are accounted for exactly once.

Comparison with the original pipeline and trade-offs. The original single-session pipeline would have treated the new scan as a replacement: H3 would have vanished without any record, H10 would have been indistinguishable from an old host, and a changed port list would not have been highlighted. The multi-session version keeps history and makes change explicit, but at a cost: more keys and more state to maintain, larger storage (every session is kept), slower merges, and the need for rules to decide when an 'unmatched' host is really gone.

Stage 6 - Joint evaluation

Why does the scan-data pipeline require a structured sequence of import, validation, cleaning, reshaping, merging and export?

Each stage depends on the guarantees of the previous one. Import guarantees columns and types; validation guarantees that every remaining row is in scope and well formed; cleaning guarantees unique keys and consistent names; reshaping is only meaningful on clean, consistently typed data; merging needs unique keys to avoid row multiplication; and export needs a final, trusted table. If validation is skipped, the out-of-scope host H4 and the invalid port 70000 would flow into the merge and into the management report. If the order is reversed (merge before de-duplication) the duplicate H1:80 would multiply rows.

How do modular functions and systematic transformations improve correctness, maintainability and efficiency?

- **Correctness:** each function is tested once on valid, invalid and boundary inputs and then reused; a fix in valid_ip() reaches every stage automatically.
- **Maintainability:** the scope pattern, required columns and keys are arguments, so moving to the 192.168.57.0/24 subnet needed no code change (Stage 1).
- **Efficiency:** vectorised functions and a single do.call(rbind, ...) removed the quadratic copying (4.8x at 5,000 records), and logs make bottlenecks visible.

What additional costs or challenges arise from validation, error handling, repeated scanning and data integration?

- Validation and error handling add code, test effort and run time, and produce rejected rows that someone must review.
- Repeated scanning multiplies stored versions (every run writes a new file) and requires a retention policy.
- Integration adds key-management risk: a wrong or unstable key silently produces missing or multiplied rows, so every join needs a count check.
- Manual review does not scale: the MANUAL queue grows with the number of scanned hosts.

Under what conditions might the proposed pipeline become inefficient, unreliable or unsuitable for changing requirements?

- **Inefficient:** when scan volume outgrows memory (millions of host-port rows in data frames) or when row-wise loops are reintroduced.
- **Unreliable:** when keys are unstable (IPs change, hostnames reused), when services have several findings per port that are merged without aggregation, or when the controlled service list is out of date.
- **Unsuitable:** when requirements change faster than the schema (new protocols, cloud assets, IPv6, ephemeral containers) and the pipeline needs the schema evolution described in Stage 5 to be applied by hand.

Conclusion - what should a security analyst consider when designing a reliable, efficient and maintainable scan-data pipeline for evolving networks?

- **Stable keys** (host_id, port, protocol) and a documented schema.
- **Explicit scope and validation first**, with rejected rows retained in an audit table.
- **Deterministic automation plus a manual-review queue** for what rules cannot decide.
- **Idempotent, reproducible stages** with record counts logged at every step.
- **Versioned, non-destructive outputs** and read-back validation.
- **Modular, tested, parameterised functions** (scope, keys and columns as arguments).
- **A schema that can evolve** (new protocols, new hosts, first_seen/last_seen) without rewriting the whole workflow, and benchmarks to know when it must be redesigned.

Contribution Record

The record reflects the actual division of primary responsibility as defined by the assignment. Primary contribution does not mean exclusive responsibility; both students participated in every question.

Task	Student A - primary contribution	Student B - primary contribution	Joint work
Q1	Host and port schemas, requirements-to-column mapping, scan-data structure justification	Relationship map for Services, OS/Versions, Enumeration; four enquiry questions; acceptance criteria	Five datasets, schema and relationship plan, acceptance criteria
Q2	Hosts and Ports/Services frames; defect injection; schema and defect explanation	Services, OS/Versions, Enumeration frames; validation and standardisation checks	Validation ordering design answer
Q3	Missing-value, duplicate and malformed-address cleaning; rule explanations	Service-name standardisation; manual-review flags; verification tests; no-silent-loss test	Reusable operations, before/after tests, auto-vs-manual matrix, joint conclusion
Q4	Wide-to-long melt; identifier/measure columns; long-format justification	Cast summaries; total comparison; preservation explanation	Wide data set, identifiers/dimensions/measures, joint conclusion
Q5	Services-to-Hosts and Ports-to-Hosts joins; key choices; count checks	OS/Versions and Enumeration integration; alternative joins; exceptions	Master table, merge-type comparison, unmatched detection
Q6	Scan-file import; schema/type checks; error demonstration	Export, naming and versioning; read-back comparison	Strategy for naming and versioning
Q7	Validation/cleaning functions; pipeline import-cleaning integration; per-host/service reports; scaling and baseline; schema evolution	Reshaping/merging/file functions; export integration and reproducibility; exposure report; bottleneck analysis; multi-session workflow	Function-library design, Stage 6 evaluation and conclusion

Signature of Student A: _____ Signature of Student B: _____

Individual Verification (Viva) Preparation

Each student must be able to answer for the whole assignment. The questions below follow the recommended three-question model: a question from the component the student led, a question from the partner's component and a transfer question.

#	Question	Model answer
1	Why is Ports keyed on (host_id, port, protocol) rather than host_id and port?	The same port number can be open on TCP and UDP; protocol makes the pair unique so that legitimate UDP and TCP services are not collapsed as duplicates.
2	Why is the scope check applied before duplicate detection?	Out-of-scope rows are an authorisation problem and invalidate downstream work; removing them first reduces the data every later check must process.
3	Which cleaning rule imputes a value, and why is it safe?	Only the missing hostname is filled with 'unknown'; it is a non-key descriptive field and cannot change a finding.
4	How do you prove cleaning did not silently drop open ports?	Compare valid, unique open ports on retained hosts computed from the raw data with those in the cleaned data (5 = 5), and show rows cleaned + rows rejected = raw rows (7 + 3 = 10).
5	What are the identifier, dimension and measured variable after melting?	Identifier host_id; dimension port; measured variable state.
6	How do you know the cast totals are right?	They equal the source counts: 5 open ports, 7 scanned pairs; three independent aggregates reconcile.
7	Why can the same two tables give 5, 7 or 8 rows?	Inner keeps matches only (5), left keeps all left rows (7), full also keeps unmatched right rows (8: H4:8080).
8	What happens if a service appears twice for the same (host_id, port)?	The join multiplies rows (7 becomes 8); the stopifnot guard fails; services must be de-duplicated first.
9	How does the pipeline avoid overwriting an earlier scan?	Timestamped file names plus a collision suffix (_01, _02) when the name already exists.
10	What does a missing 'state' column do to read_scan()?	It stops with 'missing columns: state' before any processing.
11	What was the bottleneck at 5,000 records and what fixed it?	Repeated rbind() in a loop (quadratic copying); replaced by vectorised validation and a single do.call(rbind).

#	Question	Model answer
12	Transfer: a service record exists for a host that is not in Hosts. What happens?	A left join from the host-port table never uses it, so it does not enter the master; the anti-join reports it as unmatched (as for H4:8080) for manual review.
13	Transfer: a new host appears in session 2. How is it handled?	classify_sessions() marks it newly_discovered with first_seen = new session date; it is added to the master, not merged over an old record.
14	Transfer: the number of scanned ports increases 100 times. What happens to processing time?	Row-wise code grows roughly quadratically (0.02 s to 1.9 s here); vectorised code grows roughly linearly; memory also grows and may need chunking.
15	Transfer: a UDP scan returns 'open|filtered'. How should it be treated?	Protocol joins the key; the state is accepted for UDP only and flagged for review, not counted as open in the exposure score.

Overall Conclusion

This assignment treated a scan not as a one-time command output but as data that must be modelled, validated, cleaned, reshaped, merged, versioned and reported. The five normalised datasets, the scope-first validation order, the audit table of rejected rows, the count-checked joins and the versioned read/write pipeline together make the results traceable and reproducible. The multi-session redesign and the benchmark show where the pipeline would need to evolve. The key lesson is that automation reduces effort but does not replace verification: every stage logs counts, every rejection is kept, and ambiguous records are passed to an analyst before they become findings.

Appendix A. Scan Acquisition in the Isolated Lab

The following commands were run only from the lab attacker VM against the host-only lab network 192.168.56.0/24, with the written permission of the lab owner (the students). The results were exported and converted to the CSV format read by read_scan().

```
# Host discovery (which hosts are up)
nmap -sn 192.168.56.0/24 -oG out/hosts_up.gnmap
# TCP port, service/version and OS detection on the
lab range
nmap -sS -sV -O -p- 192.168.56.0/24 -oX
out/session1.xml -oG out/session1.gnmap
# UDP top ports (Stage 5 change requirement)
nmap -sU --top-ports 50 192.168.56.0/24 -oX
out/session2_udp.xml
# Optional enumeration scripts on discovered
services (lab only)
nmap -sV --script "default and safe" 192.168.56.12 -
p 21,445
```

The XML or grepable output is converted to the columns host_id, ip, port, protocol, state (plus service_name and version) before import. The sample datasets in this report reproduce the structure of such an export.